Universidad Tecnológica Nacional

Facultad Regional Córdoba

Licenciatura en Automatización y Control

Informática Industrial

TRABAJO PRÁCTICO Nº 3 — GRUPO Nº 4

Diseño, implementación y consulta de una base de datos relacional para la Gestión de Mantenimiento Industrial (GMAO)

Informe técnico

Docentes

Ing. Carolina Rivarola

Alumnos

Casali, Emiliano

Del Soto, Gustavo

Ferrero, Facundo

Vera, Gonzalo

Córdoba · Octubre de 2026

Índice

Introducción 3
- Objetivo 3
- Caso de estudio 3
- Alcance 4
- Metodología y organización del informe 4

Fase I — Fundamentos y clasificación (La Investigación) 5
- Paso 1 — Definición y usos industriales de las bases de datos 5
- Paso 2 — Selección del motor de base de datos y requerimientos 8

Fase II — Diseño conceptual y modelado (La Planificación) 14
- Paso 3 — Identificación de entidades y atributos 14
- Paso 4 — Construcción del modelo relacional 20

Fase III — Implementación y carga de datos mediante SQL (La Construcción) 24
- Paso 5 — Scripts de creación de tablas (DDL) 24
- Paso 6 — Script de inserción de datos de planta (DML) 30

Fase IV — Explotación de datos y reportes industriales (La Consulta) 38
- Paso 7 — Consultas de selección y filtrado básico 38
- Paso 8 — Consultas de agrupación y estadísticas 42
- Paso 9 — Consultas de integración de tablas (JOINs) 45

Referencias 48

Introducción

En una planta industrial, cada hora que una máquina está detenida es producción que no se entrega. El área de mantenimiento existe para reducir esas horas: repara lo que falla y, sobre todo, se anticipa a las fallas con tareas preventivas. Para hacerlo bien necesita información confiable. Necesita saber qué máquinas fallan más, cuánto tiempo estuvieron paradas, qué órdenes de trabajo siguen abiertas, quién las atiende y qué repuestos faltan en el pañol. Cuando esa información vive en planillas sueltas o en cuadernos, se pierde, se duplica o se contradice. Un sistema de **gestión de mantenimiento asistido por ordenador (GMAO)** la centraliza, y en su base hay una base de datos relacional.

Objetivo

El objetivo de este trabajo es investigar, diseñar, implementar y consultar una base de datos relacional orientada al ámbito industrial, usando el lenguaje SQL. De los escenarios propuestos por la cátedra se eligió el de **Gestión de Mantenimiento Industrial (GMAO)**: registro de activos, historial de fallas, órdenes de trabajo correctivas y preventivas, técnicos asignados y repuestos utilizados.

Caso de estudio

Para que los datos tengan sentido se planteó una empresa: **Autopartes G4 S.A.**, una metalmecánica ficticia de Córdoba que fabrica autopartes para proveedores de la industria automotriz y trabaja en tres turnos. Su proceso productivo combina cuatro líneas y un servicio común:

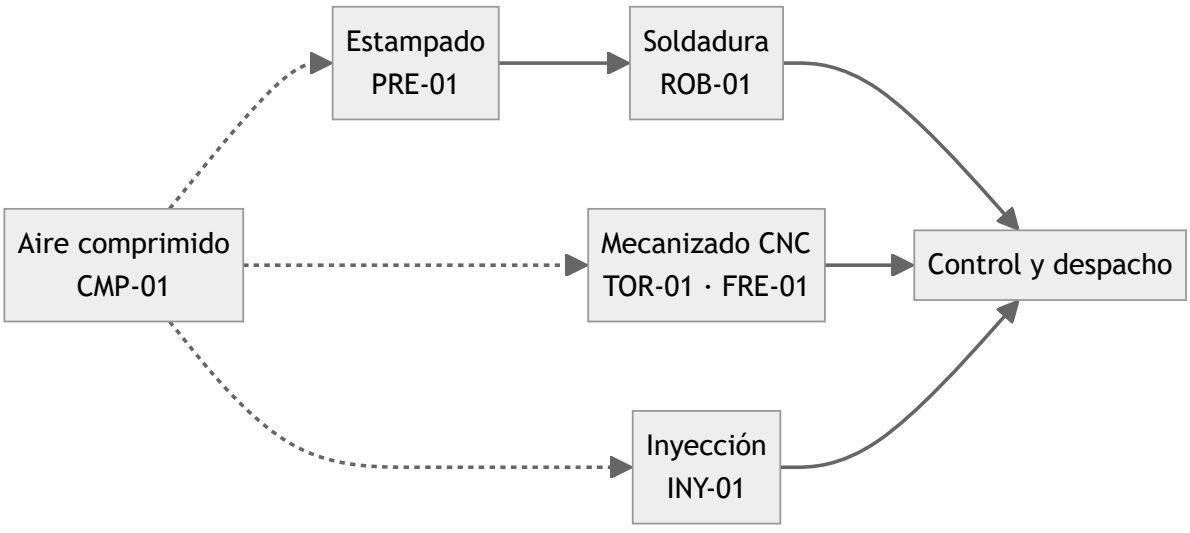


Figura 1. Proceso productivo de la planta del caso de estudio y máquinas bajo mantenimiento

La prensa hidráulica estampa la chapa, el robot suelda los subconjuntos, el torno CNC y el centro de mecanizado fabrican piezas de precisión y la inyectora produce los componentes plásticos. El compresor abastece de aire comprimido a toda la planta. Esas seis máquinas son los activos que gestiona el sistema. Un equipo de cinco técnicos de distintas especialidades los mantiene, con un pañol de repuestos propio. Las fallas dan lugar a órdenes de trabajo correctivas, y un conjunto de planes preventivos, por calendario o por horas de uso, genera las órdenes preventivas.

El nombre de la empresa, el personal y todos los registros son **simulados**. Las marcas y modelos de las máquinas son reales y se eligieron para que los datos sean verosímiles. El período registrado va de septiembre a octubre de 2026.

Alcance

El sistema cubre el registro de los activos y su historial de fallas, los planes preventivos, las órdenes de trabajo con su técnico, su estado y sus horas-hombre, y el consumo y el stock de repuestos. Quedan fuera las compras, el costo de la mano de obra, la programación automática del preventivo y la lectura automática de sensores.

Metodología y organización del informe

El trabajo se desarrolló en forma secuencial, en las cuatro fases que fija el enunciado. Cada paso incluye su justificación teórica, la descripción del procedimiento y capturas de pantalla como evidencia.

- **Fase I — La Investigación (Pasos 1 y 2).** Qué es una base de datos, cómo se clasifican y para qué se usan en la industria. Elección del motor **SQLite** y de la herramienta **DB Browser for SQLite**.
- **Fase II — La Planificación (Pasos 3 y 4).** Las entidades del mantenimiento, sus atributos y tipos de datos, y el modelo relacional con su diagrama entidad-relación. Antes de definir el modelo se relevó cómo resuelven el problema los gestores de mantenimiento de uso industrial.
- **Fase III — La Construcción (Pasos 5 y 6).** Los scripts que crean las tablas con sus restricciones de integridad y los que cargan los datos de planta.
- **Fase IV — La Consulta (Pasos 7, 8 y 9).** Siete consultas que responden preguntas concretas del supervisor, del encargado del pañol y del jefe de mantenimiento: filtrado, estadísticas por agrupación y reportes que cruzan tablas.

El código SQL aparece en el texto en letra monoespaciada y es el mismo que se ejecutó en las capturas. El conjunto completo de sentencias se entrega, además, en el archivo `tp3-gmao.sql`. Las fuentes consultadas se listan en las referencias, al final del informe.

Fase I — Fundamentos y clasificación (La Investigación)

En esta fase se investiga qué es una base de datos, cómo se clasifican y para qué se usan las bases relacionales en una planta industrial (Paso 1). Con esa base se elige y se justifica el gestor de base de datos del trabajo, se detallan sus requisitos de instalación y se describe cómo se accede a la base en forma local (Paso 2).

Paso 1 — Definición y usos industriales de las bases de datos

Qué es una base de datos

Una **base de datos** es un conjunto de datos relacionados entre sí, organizados con una estructura definida y guardados de forma permanente, para que distintas personas y programas puedan consultarlos y actualizarlos sin perder coherencia. La diferencia con un archivo suelto o una planilla es que la estructura y las reglas que deben cumplir los datos son parte de la propia base, y no dependen del cuidado de quien los carga.

El software que administra la base es el **sistema gestor de bases de datos** (SGBD, o DBMS en inglés). Es el que define la estructura, controla quién accede, hace cumplir las reglas de integridad y resuelve las consultas.

El modelo más difundido es el **relacional**, propuesto por E. F. Codd en 1970 [1]. En él los datos se organizan en **tablas**: cada fila es un registro y cada columna, un atributo. Las tablas se vinculan por medio de **claves**. La clave primaria (PK) identifica cada fila, y la clave foránea (FK) guarda la clave de otra tabla y crea el vínculo [2]. Para definir, cargar y consultar los datos se usa el lenguaje estándar **SQL** [2].

Las bases relacionales trabajan con **transacciones ACID**, que garantizan que los datos queden correctos aunque haya errores o cortes [2]:

- **Atomicidad:** una transacción se ejecuta completa o no se ejecuta.
- **Consistencia:** después de cada transacción los datos siguen cumpliendo todas las reglas definidas.
- **Aislamiento:** las transacciones simultáneas no se interfieren entre sí.
- **Durabilidad:** lo que se confirma queda guardado, aun ante un corte de energía.

Clasificación de las bases de datos

Según cómo organizan los datos, las bases se agrupan en tres grandes familias, relevantes para la industria.

Bases relacionales (SQL). Tienen un esquema fijo, definido antes de cargar los datos, y vinculan las tablas por claves. Ofrecen consistencia fuerte (ACID) y permiten cruzar información de varias tablas en una sola consulta (`JOIN`). Crecen principalmente en forma **vertical**, con un servidor más potente [3]. Ejemplos: SQLite, PostgreSQL, MySQL, SQL Server y Oracle.

Bases NoSQL (*not only SQL*). Guardan los datos fuera del modelo de tablas, en general sin un esquema fijo, y suelen estar distribuidas en varios servidores, por lo que crecen en forma **horizontal** [3]. Hay cuatro tipos principales [3]:

- **documentales**, que guardan documentos JSON (por ejemplo, MongoDB);
- **clave-valor**, usadas como memoria caché (por ejemplo, Redis);
- **de columnas amplias**, para grandes volúmenes de datos dispersos (por ejemplo, Cassandra);
- **de grafos**, para redes de relaciones (por ejemplo, Neo4j).

Su punto débil son las transacciones complejas y las relaciones entre entidades, que son más difíciles de garantizar que en una base relacional [3].

Bases de series temporales (TSDB). Están diseñadas para mediciones con marca de tiempo que llegan en forma continua, como las de sensores. Admiten un volumen de escritura muy alto y resuelven rápido las consultas por rangos de tiempo. A cambio, modificar o borrar un dato puntual es lento, algo que en este tipo de datos casi no ocurre [4]. Incluyen políticas de retención y comprimen mucho los datos [4]. Los **historiadores de proceso** de los sistemas SCADA son una forma de base de series temporales [5]. Ejemplos: InfluxDB, TimescaleDB y los historiadores industriales.

Tabla 1. Clasificación de las bases de datos

Criterio	Relacional	NoSQL	Series temporales
Modelo	Tablas vinculadas por claves	Documentos, clave-valor, columnas o grafos	Mediciones con marca de tiempo
Esquema	Fijo, definido de antemano	Flexible o sin esquema	Flexible, centrado en el tiempo
Consistencia	Fuerte (ACID)	Variable según el motor	Orientada a la escritura masiva
Crecimiento	Vertical	Horizontal	Horizontal, con compresión
Consulta típica	Cruces entre tablas y reportes	Lectura por clave o documento	Rangos de tiempo, promedios y tendencias
Uso en planta	ERP, MES, GMAO, recetas, calidad	Configuraciones, registros de eventos, datos heterogéneos	Variables de proceso, sensores, historiador SCADA

En una fábrica estas familias no compiten: se **complementan**. Lo habitual es una arquitectura híbrida, con una base relacional para las entidades de gestión (órdenes, equipos, materiales, personal) y una base de series temporales para las mediciones de alta frecuencia [4].

Tres usos críticos de las bases relacionales en la fábrica

El estándar **ISA-95** (IEC 62264) organiza la empresa industrial en cinco niveles, desde el proceso físico hasta la gestión del negocio [6]. Sirve para ubicar dónde se usa cada tipo de base de datos:

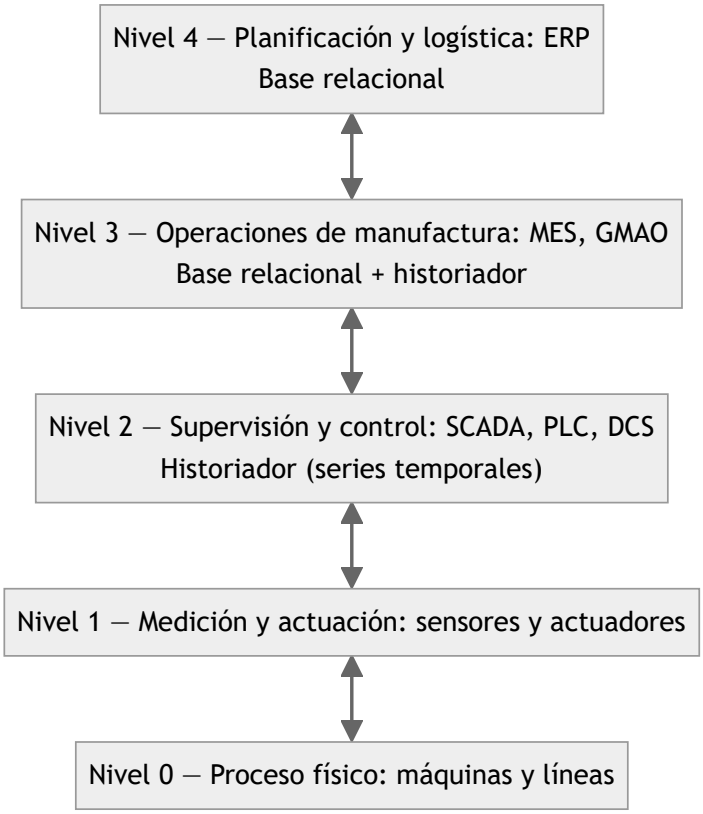


Figura 2. Niveles del modelo ISA-95 y tipo de base de datos predominante en cada uno

Los niveles 3 y 4, donde se gestionan órdenes, equipos, materiales y personas, son el terreno de las bases relacionales. Tres usos críticos son los siguientes.

Uso 1 — Integración entre el ERP y el MES

El ERP (nivel 4) planifica qué se produce, y el MES (nivel 3) gestiona cómo se produce en planta. ISA-95 define el intercambio entre esos dos niveles y los modelos de datos de equipos, materiales, personal y órdenes que comparten, que son la base para diseñar bases relacionales [6][7]. La base relacional guarda las órdenes de producción que bajan del ERP, la producción real y los consumos que suben del MES, y los maestros de materiales y equipos.

Es un uso crítico porque cada dato tiene que ser coherente con los demás: una orden no puede hacer referencia a un material inexistente, y un consumo no puede registrarse a medias. La integridad referencial y las transacciones ACID garantizan las dos cosas. Además, el nivel 3 actúa como capa intermedia: el ERP es un sistema transaccional y no está pensado para recibir datos de sensores en tiempo real [7].

Uso 2 — Almacenamiento y versionado de recetas de proceso

En los procesos por lotes (alimentos, química, farmacia), el estándar **ISA-88** define las recetas en cuatro niveles: general, de sitio, maestra y de control. Cada receta tiene un encabezado, una fórmula con parámetros y materiales, los requerimientos de equipo y el procedimiento [8]. La **receta maestra** es la aprobada para un conjunto de equipos, y la **receta de control** es la copia que se ejecuta en un lote concreto, con sus valores reales [8].

Una receta es información estructurada y relacionada: pasos, parámetros, consignas y materiales. Guardarla en una base relacional permite controlar sus versiones y su estado de aprobación, impedir que se ejecute una receta no aprobada y registrar qué versión se usó en cada lote, lo que da trazabilidad ante un reclamo de calidad.

Uso 3 — Gestión del mantenimiento (GMAO)

Un sistema de **gestión de mantenimiento asistido por ordenador** (GMAO, o CMMS en inglés) centraliza la información de mantenimiento de la planta [9]:

- el registro de activos y su historial de intervenciones;
- las órdenes de trabajo, desde que se emiten hasta que se cierran, con el técnico asignado;
- los planes de mantenimiento preventivo, que se disparan por tiempo o por uso;
- el stock de repuestos y su reposición cuando baja del mínimo.

Son entidades bien definidas y muy relacionadas: una máquina tiene fallas, las fallas generan órdenes de trabajo, las órdenes se asignan a técnicos y consumen repuestos. Los reportes que necesita el área de mantenimiento cruzan esas tablas y calculan indicadores, como las fallas por máquina, el tiempo de parada o las horas-hombre. Ese es exactamente el tipo de problema que resuelve una base relacional, y es el **escenario elegido para este trabajo**, que se desarrolla en las Fases II a IV.

Paso 2 — Selección del motor de base de datos y requerimientos

Elección del gestor: SQLite

Para el trabajo se eligió **SQLite**. Sus características, según la documentación oficial, son las siguientes [10]:

- **Sin servidor:** es una biblioteca que corre dentro del programa que la usa y lee y escribe directamente sobre un archivo. No hay un servicio que instalar, configurar ni administrar.
- **Un solo archivo:** la base completa, con tablas, índices y *triggers*, queda en un único archivo `.db`, fácil de copiar y de entregar.
- **Transacciones ACID**, aun ante caídas del sistema o cortes de energía.
- **Claves foráneas, restricciones `CHECK` y *triggers***, que son las herramientas de integridad que necesita el modelo GMAO [15].

- **Dominio público:** se puede usar libremente con cualquier fin.
- **Multiplataforma y liviano:** el mismo archivo funciona en Windows, Linux y macOS, y la biblioteca completa ocupa menos de 900 KiB.

El propio proyecto SQLite lo recomienda para dispositivos embebidos, para análisis de datos y para **educación**, porque se instala en forma trivial y las bases se crean y se comparten con facilidad [11]. Es el caso de este trabajo práctico.

También tiene límites, que hay que conocer [11]. Admite **un solo escritor a la vez**, no conviene usarlo a través de una red y, para volúmenes del orden de los terabytes, es preferible un motor cliente-servidor. Por eso, un GMAO multiusuario en una planta real correría sobre PostgreSQL, SQL Server o MySQL. Para diseñar, construir y probar el modelo, SQLite alcanza, y el SQL usado es estándar y se puede trasladar a esos motores.

Tabla 2. Elección del gestor: SQLite

Criterio	SQLite	Cliente-servidor (PostgreSQL, MySQL, SQL Server)
Arquitectura	Biblioteca embebida, archivo local	Servicio en un servidor, acceso por red
Instalación	Nula: el motor viene incluido en la herramienta	Instalar y configurar un servicio, usuarios y permisos
Escritura simultánea	Un escritor a la vez	Muchos escritores simultáneos
Entrega del trabajo	Un archivo `.db`	Exportar un volcado de la base
Uso típico	Embebido, IoT, educación, prototipos	Sistemas de planta multiusuario

Herramienta de gestión: DB Browser for SQLite

Como interfaz gráfica se usa **DB Browser for SQLite**, una herramienta visual de código abierto para crear, editar y consultar bases SQLite [12]. Las licencias publicadas en su repositorio son GPL-3.0 y MPL-2.0 [13]. Funciona en Windows, macOS y Linux [12]. Para este trabajo aporta [12]:

- la creación de la base y la ejecución de scripts SQL completos, con su panel de mensajes (Pasos 5 a 9);
- la vista de la estructura de la base, con tablas, columnas y *triggers* (Pasos 4 y 5);
- la navegación de los registros de cada tabla (Paso 6);
- la exportación de la base como script SQL, útil para el respaldo.

Requerimientos de instalación

La versión estable de DB Browser for SQLite es la **3.13.1** [13][14]. Para Windows se ofrece como instalador `.msi` y como `.zip` sin instalación, en versiones de 32 bits, 64 bits y ARM64, además de una versión portable; los ejecutables están firmados digitalmente [14]. También se puede instalar con gestores de paquetes como `winget` [13]. **No hace falta instalar SQLite por separado**, porque la herramienta trae el motor incluido. El sitio oficial no publica requisitos mínimos de hardware [14], de modo que se documentan los del equipo donde se hizo el trabajo:

Tabla 3. Requerimientos de instalación

Elemento	Valor
Sistema operativo	Windows 11 Pro, 64 bits (versión 10.0.26200)
Procesador	AMD Ryzen 5 5500
Memoria RAM	16 GB
DB Browser for SQLite	3.13.1, instalador `.msi` de 64 bits
Motor SQLite incluido	3.46.1
Espacio ocupado por la instalación	47 MB
Carpeta de instalación	`C:\Program Files\DB Browser for SQLite\`

La instalación se hizo con el gestor de paquetes de Windows:

```
winget install --id DBBrowserForSQLite.DBBrowserForSQLite -e
```



Figura 3. Ventana Ayuda → Acerca de: DB Browser for SQLite 3.13.1 con SQLite 3.46.1

Método de acceso local

Como SQLite no tiene servidor, acceder a la base es abrir su archivo. El procedimiento en DB Browser for SQLite es el siguiente:

1. Abrir DB Browser for SQLite y elegir **Nueva base de datos**. Indicar la carpeta y el nombre del archivo: `gmao.db`. Si se abre el asistente para crear una tabla, cerrarlo, porque las tablas se crean por código en el Paso 5.
2. En **Editar** → **Preferencias** → **Base de datos**, verificar que esté activada la opción de **claves foráneas**. En SQLite la verificación de claves foráneas viene desactivada por defecto y se activa en cada conexión con `PRAGMA foreign_keys = ON` [15]. Los scripts del trabajo incluyen esa sentencia al comienzo, para no depender de la configuración.
3. En la pestaña **Ejecutar SQL**, escribir o abrir un script y ejecutarlo con **Ejecutar todo** (`F5`). El panel inferior informa si la ejecución terminó sin errores.
4. Confirmar los cambios con **Guardar cambios** (`Ctrl+S`). Hasta ese momento los cambios quedan en una transacción abierta y se pierden si se cierra el programa.
5. Revisar el resultado en las pestañas **Estructura** (tablas, columnas y *triggers*) y **Hoja de datos** (registros de cada tabla).
6. Para volver a trabajar, abrir el mismo archivo con **Abrir base de datos**.

Para comprobar que las claves foráneas están activas, se ejecuta:

```
PRAGMA foreign_keys;
```

El resultado `1` indica que están activadas, y `0`, que no.

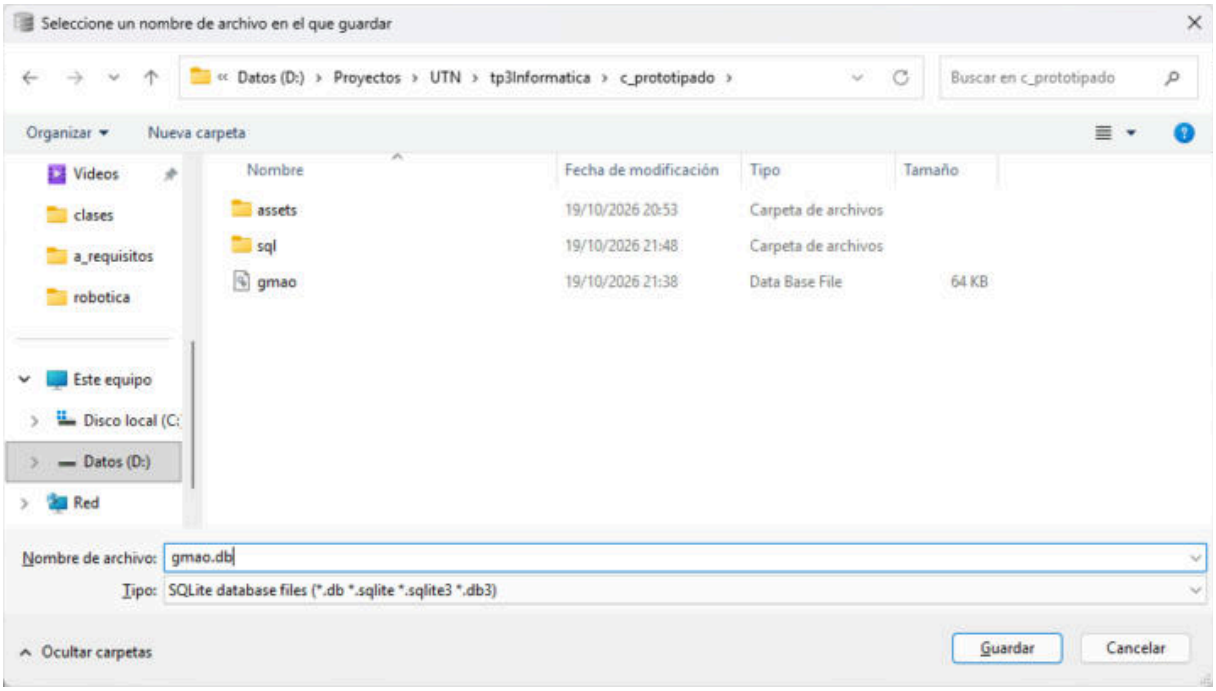


Figura 4. Cuadro de diálogo de Nueva base de datos con el archivo gmao.db en la carpeta del proyecto

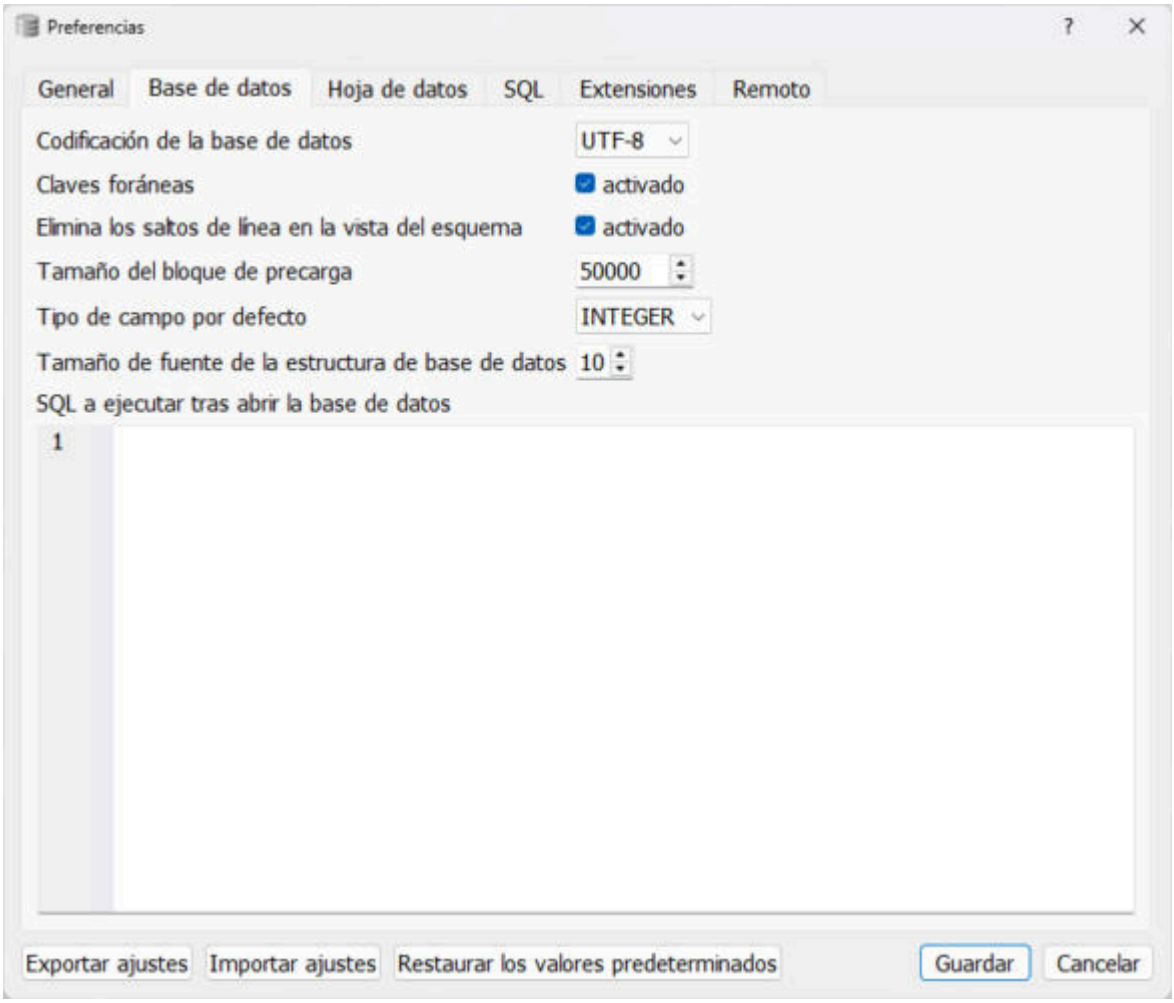


Figura 5. Editar → Preferencias → Base de datos, con la opción Claves foráneas activada

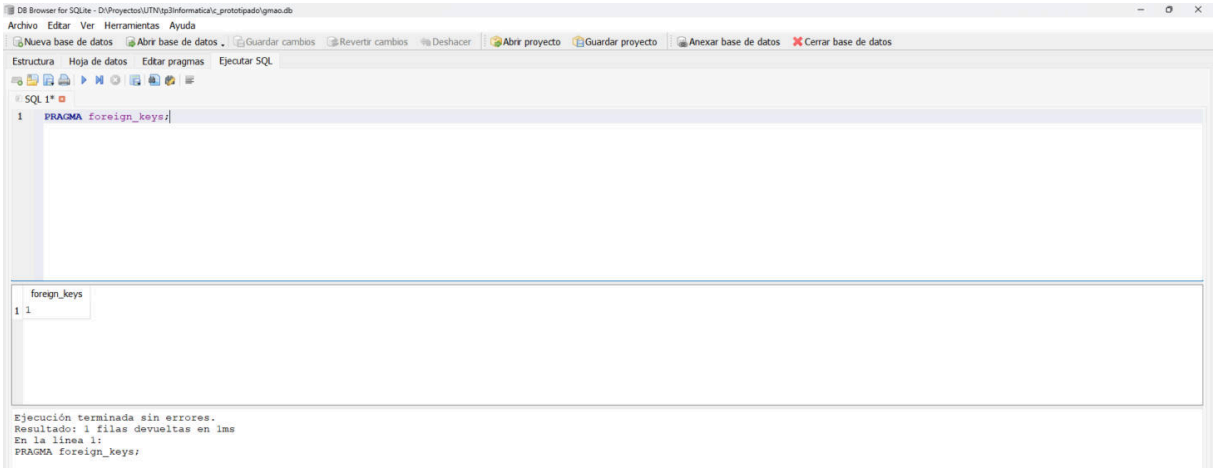


Figura 6. Pestaña Ejecutar SQL: PRAGMA foreign_keys devuelve 1, las claves foráneas están activadas

Fase II — Diseño conceptual y modelado (La Planificación)

En esta fase se traduce el escenario de Gestión de Mantenimiento Industrial (GMAO) a un modelo de datos. Primero se identifican las entidades del proceso de mantenimiento y sus atributos, con el tipo de dato adecuado para cada uno (Paso 3). Después se arma el modelo relacional con sus claves primarias, claves foráneas y cardinalidades (Paso 4). El resultado de esta fase es la base de los scripts de la Fase III.

Paso 3 — Identificación de entidades y atributos

Justificación teórica

Una **entidad** es un objeto del mundo real del que interesa guardar información y que se distingue de los demás: una máquina, un técnico, una falla. Cada entidad se describe con **atributos**, y cada atributo toma valores de un **dominio**, que en la base de datos se expresa con un **tipo de dato**. Elegir bien el tipo de dato no es un detalle: define qué valores se aceptan, cuánto espacio ocupan, cómo se ordenan y qué operaciones se pueden hacer con ellos. Una variable analógica guardada como texto, por ejemplo, no se puede promediar.

Tipos de datos en SQLite

En SQLite el tipado es dinámico: el tipo pertenece al valor y no a la columna. Cada columna tiene una **afinidad** que se deduce del tipo declarado y que indica la clase de almacenamiento preferida (`INTEGER`, `REAL`, `TEXT`, `BLOB` o `NUMERIC`). Por eso se pueden declarar los tipos industriales habituales (`INT`, `VARCHAR`, `FLOAT`, `DATETIME`) y SQLite los acepta y los asigna a su afinidad:

Tabla 4. Tipos de datos en SQLite

Tipo declarado	Afinidad en SQLite	Uso en este modelo
`INTEGER`	INTEGER	Identificadores y cantidades enteras (stock, unidades consumidas, frecuencias).
`VARCHAR(n)`	TEXT	Nombres, códigos, descripciones y estados. SQLite no limita el largo `n`, pero se declara para documentar el tamaño esperado.
`FLOAT`	REAL	Variables analógicas y continuas: horas de operación, horas de parada, horas-hombre, costos.
`DATE`	NUMERIC	Fechas sin hora (instalación de un equipo).
`DATETIME`	NUMERIC	Marcas de tiempo de eventos (falla, emisión y cierre de una orden).

SQLite no tiene un tipo propio para fechas. Se adopta el formato de texto **ISO 8601** (`AAAA-MM-DD HH:MM:SS`), que se ordena correctamente como texto, se compara con `BETWEEN` y se puede procesar con las funciones de fecha de SQLite.

Los identificadores se declaran `INTEGER PRIMARY KEY`. En SQLite esa combinación convierte la columna en alias del identificador interno de cada fila (`rowid`), lo que la hace única, no nula y autoincremental sin configuración adicional.

Procedimiento

Para definir las entidades se relevó cómo resuelven el mantenimiento los gestores de uso industrial, en particular SAP PM, Odoo Maintenance e IBM Maximo. Los tres coinciden en que el mantenimiento **correctivo** y el **preventivo** tienen orígenes distintos y terminan en el mismo documento de ejecución, la orden de trabajo:

1. **Correctivo.** Una **máquina** (activo) presenta una **falla**, que se registra con su fecha, hora, descripción y tiempo de parada. A partir de la falla se emite una **orden de trabajo** correctiva. Mientras no se emite la orden, la falla queda pendiente. Una misma orden puede atender varias fallas detectadas en la misma intervención, como en el esquema aviso–orden de SAP PM.
2. **Preventivo.** El área de mantenimiento preventivo define, para cada máquina, **planes preventivos**: una tarea con su frecuencia, por calendario (cada N días) o por uso (cada N horas de operación del horómetro). Cuando el plan vence, se emite una **orden de trabajo** preventiva. En el preventivo **no existe falla**: la orden nace del plan.
3. En los dos casos la orden se asigna a un **técnico** responsable y registra fechas, estado y horas-hombre.
4. Durante el trabajo se consumen **repuestos** del almacén, y una misma orden puede usar varios repuestos distintos.

De ese recorrido surgen seis entidades: las cinco que nombra el enunciado para GMAO (activos, fallas, órdenes de trabajo, técnicos y repuestos) más los **planes preventivos**, que son el origen de las órdenes preventivas. Una séptima tabla, `ot_repuestos`, registra qué repuestos y en qué cantidad se consumieron en cada orden: una orden usa varios repuestos y un repuesto se usa en varias órdenes, y esa relación de muchos a muchos se resuelve en el Paso 4.

Para cada atributo se definió el tipo de dato y las restricciones que se aplicarán en el Paso 5: clave primaria (PK), clave foránea (FK), obligatorio (`NOT NULL`), único (`UNIQUE`) o con valores permitidos (`CHECK`).

Estructura de la tabla `maquinas`

Tabla 5. Estructura de la tabla maquinas

Campo	Tipo	Restricciones	Descripción
`id_maquina`	`INTEGER`	PK	Identificador interno.
`codigo`	`VARCHAR(10)`	`NOT NULL`, `UNIQUE`	Código o TAG del activo en planta (p. ej. `TOR-01`).
`nombre`	`VARCHAR(60)`	`NOT NULL`	Nombre descriptivo del equipo.
`marca`	`VARCHAR(40)`	—	Fabricante.
`modelo`	`VARCHAR(40)`	—	Modelo del fabricante.
`sector`	`VARCHAR(40)`	`NOT NULL`	Sector o línea donde está instalado.
`fecha_instalacion`	`DATE`	—	Fecha de puesta en servicio.
`horas_operacion`	`FLOAT`	—	Horómetro acumulado, en horas.
`criticidad`	`VARCHAR(5)`	`NOT NULL`, `CHECK (ALTA, MEDIA, BAJA)`	Impacto de su parada en la producción.

Registro de activos de la planta: cada equipo sujeto a mantenimiento.

Estructura de la tabla `tecnicos`

Tabla 6. Estructura de la tabla tecnicos

Campo	Tipo	Restricciones	Descripción
`id_tecnico`	`INTEGER`	PK	Identificador interno.
`legajo`	`VARCHAR(10)`	`NOT NULL`, `UNIQUE`	Número de legajo del empleado.
`nombre`	`VARCHAR(40)`	`NOT NULL`	Nombre.
`apellido`	`VARCHAR(40)`	`NOT NULL`	Apellido.
`especialidad`	`VARCHAR(20)`	`NOT NULL`	Mecánica, eléctrica, hidráulica, electrónica.
`turno`	`VARCHAR(10)`	`NOT NULL`, `CHECK (MAÑANA, TARDE, NOCHE)`	Turno de trabajo.
`telefono`	`VARCHAR(20)`	—	Teléfono interno o celular de guardia.

Personal de mantenimiento al que se asignan las órdenes.

Estructura de la tabla `planes_preventivos`

Tabla 7. Estructura de la tabla planes_preventivos

Campo	Tipo	Restricciones	Descripción
`id_plan`	`INTEGER`	PK	Identificador interno.
`codigo`	`VARCHAR(15)`	`NOT NULL`, `UNIQUE`	Código del plan (p. ej. `PM-CMP01-ACE`).
`id_maquina`	`INTEGER`	FK → `maquinas`, `NOT NULL`	Equipo al que se aplica el plan.
`tarea`	`VARCHAR(200)`	`NOT NULL`	Rutina a ejecutar (p. ej. cambio de aceite y filtro).
`frecuencia`	`INTEGER`	`NOT NULL`, `CHECK` (> 0)	Cada cuánto se repite la tarea.
`unidad_frecuencia`	`VARCHAR(5)`	`NOT NULL`, `CHECK` (`DIAS`, `HORAS`)	Por calendario (días) o por uso (horas de operación).
`especialidad`	`VARCHAR(20)`	`NOT NULL`	Especialidad del técnico que la ejecuta.
`duracion_estimada_h`	`FLOAT`	—	Duración prevista de la tarea, en horas.

Rutinas del área de mantenimiento preventivo: cada una da origen a órdenes preventivas periódicas.

Estructura de la tabla `ordenes_trabajo`

Tabla 8. Estructura de la tabla ordenes_trabajo

Campo	Tipo	Restricciones	Descripción
`id_ot`	`INTEGER`	PK	Identificador interno.
`nro_ot`	`VARCHAR(15)`	`NOT NULL`, `UNIQUE`	Número de orden visible en planta (p. ej. `OT-2026-0101`).
`tipo`	`VARCHAR(12)`	`NOT NULL`, `CHECK` (`CORRECTIVA`, `PREVENTIVA`)	Tipo de mantenimiento.
`id_maquina`	`INTEGER`	FK → `maquinas`, `NOT NULL`	Equipo intervenido.
`id_plan`	`INTEGER`	FK → `planes_preventivos`	Plan que originó la orden. Obligatorio en las preventivas y vacío (`NULL`) en las correctivas.
`id_tecnico`	`INTEGER`	FK → `tecnicos`, `NOT NULL`	Técnico asignado.
`fecha_emision`	`DATETIME`	`NOT NULL`	Fecha y hora de emisión.
`fecha_cierre`	`DATETIME`	—	Fecha y hora de cierre. Vacío mientras la orden sigue abierta.
`estado`	`VARCHAR(10)`	`NOT NULL`, `CHECK` (`PENDIENTE`, `EN CURSO`, `CERRADA`)	Estado de la orden.
`tarea`	`VARCHAR(200)`	`NOT NULL`	Trabajo a realizar o realizado.
`horas_hombre`	`FLOAT`	—	Horas-hombre insumidas.

Órdenes de trabajo (OT) correctivas y preventivas. Un `CHECK` de tabla exige que el origen corresponda al tipo: `(tipo = 'PREVENTIVA' AND id_plan IS NOT NULL) OR (tipo = 'CORRECTIVA' AND id_plan IS NULL)`.

Estructura de la tabla `fallas`

Tabla 9. Estructura de la tabla fallas

Campo	Tipo	Restricciones	Descripción
`id_falla`	`INTEGER`	PK	Identificador interno.
`id_maquina`	`INTEGER`	FK → `maquinas`, `NOT NULL`	Equipo que falló.
`id_ot`	`INTEGER`	FK → `ordenes_trabajo`	Orden correctiva que la atiende. Vacío (`NULL`) mientras la falla está pendiente.
`fecha_hora`	`DATETIME`	`NOT NULL`	Marca de tiempo del evento (ISO 8601).
`descripcion`	`VARCHAR(200)`	`NOT NULL`	Descripción del síntoma o del daño.
`tipo_falla`	`VARCHAR(15)`	`NOT NULL`	Mecánica, eléctrica, hidráulica, neumática, electrónica.
`severidad`	`VARCHAR(5)`	`NOT NULL`, `CHECK` (`ALTA`, `MEDIA`, `BAJA`)	Gravedad de la falla.
`horas_parada`	`FLOAT`	—	Tiempo que el equipo estuvo detenido, en horas.

Historial de fallas: cada evento que dejó un equipo fuera de servicio o fuera de condición. Con `fecha_hora` y `horas_parada` se pueden calcular indicadores de mantenimiento como el tiempo medio de reparación (MTTR).

Estructura de la tabla `repuestos`

Tabla 10. Estructura de la tabla repuestos

Campo	Tipo	Restricciones	Descripción
`id_repuesto`	`INTEGER`	PK	Identificador interno.
`codigo`	`VARCHAR(15)`	`NOT NULL`, `UNIQUE`	Código de almacén o del fabricante.
`descripcion`	`VARCHAR(100)`	`NOT NULL`	Descripción del repuesto.
`marca`	`VARCHAR(40)`	—	Fabricante.
`stock_actual`	`INTEGER`	`NOT NULL`	Unidades disponibles.
`stock_minimo`	`INTEGER`	`NOT NULL`	Punto de reposición: por debajo de este valor el stock es crítico.
`costo_unitario`	`FLOAT`	—	Costo por unidad, en pesos.

Catálogo y stock de repuestos del almacén de mantenimiento.

Estructura de la tabla `ot_repuestos`

Tabla 11. Estructura de la tabla ot_repuestos

Campo	Tipo	Restricciones	Descripción
`id_ot`	`INTEGER`	PK, FK → `ordenes_trabajo`	Orden en la que se consumió.
`id_repuesto`	`INTEGER`	PK, FK → `repuestos`	Repuesto consumido.
`cantidad`	`INTEGER`	`NOT NULL`, `CHECK` (> 0)	Unidades consumidas.

Consumo de repuestos por orden de trabajo (tabla intermedia). Su clave primaria es **compuesta** (`id_ot`, `id_repuesto`): un mismo repuesto se registra una sola vez por orden, con la cantidad total consumida.

Paso 4 — Construcción del modelo relacional

Justificación teórica

El **modelo relacional** representa los datos como un conjunto de tablas (relaciones) vinculadas entre sí por sus claves:

- La **clave primaria (PK)** identifica de forma única cada fila de una tabla. No puede repetirse ni quedar vacía.
- La **clave foránea (FK)** es una columna que guarda el valor de la PK de otra tabla y crea el vínculo entre ambas. La **integridad referencial** garantiza que toda FK apunte a una fila que existe: no puede haber una falla de una máquina que no está registrada. Una FK que admite `NULL` representa una relación **opcional**; SQLite solo verifica la referencia cuando la FK tiene un valor.
- La **cardinalidad** indica cuántas filas de una tabla se relacionan con cuántas de la otra: uno a uno (1:1), uno a muchos (1:N) o muchos a muchos (N:M). En una relación 1:N la FK va en la tabla del lado "muchos". Las relaciones N:M no se pueden representar directamente con una FK y se resuelven con una **tabla intermedia** cuya PK combina las claves de ambas tablas.

El diseño se ajusta a la **tercera forma normal (3FN)**: cada atributo depende solo de la clave de su tabla. Por ejemplo, el nombre de la máquina se guarda una sola vez en `maquinas`, y las demás tablas la referencian por `id_maquina`. Así se evitan datos repetidos y las inconsistencias que aparecen al modificarlos en un lugar y no en otro.

Procedimiento

1. Se identificaron las relaciones entre las entidades del Paso 3 a partir de las reglas del proceso de mantenimiento.
2. Para cada relación 1:N se colocó la FK en la tabla del lado "muchos".

3. La relación N:M entre órdenes de trabajo y repuestos se resolvió con la tabla intermedia `ot_repuestos`.
4. Se definieron las reglas que relacionan el tipo de orden con su origen y se decidió cómo hacerlas cumplir en la base.
5. Se dibujó el diagrama entidad-relación (DER) con las PK, FK y cardinalidades.
6. Se verificó el modelo en SQLite con un prototipo: se crearon las tablas, se cargaron casos válidos (que la base acepta) y casos inválidos (que la base debe rechazar), y se comprobó que cada regla se cumple.

Relaciones del modelo

Tabla 12. Relaciones del modelo

Relación	Cardinalidad	Implementación	Regla del proceso
`maquinas → fallas`	1:N	FK `fallas.id_maquina`	Una máquina acumula muchas fallas; cada falla pertenece a una sola máquina.
`maquinas → planes_preventivos`	1:N	FK `planes_preventivos.id_maquina`	Una máquina tiene varios planes (aceite, correas, inspecciones); cada plan es de una máquina.
`maquinas → ordenes_trabajo`	1:N	FK `ordenes_trabajo.id_maquina`	Una máquina recibe muchas OT, correctivas y preventivas.
`ordenes_trabajo → fallas`	1:N (opcional)	FK `fallas.id_ot`, admite `NULL`	Una OT correctiva atiende una o más fallas; una falla sin OT está pendiente.
`planes_preventivos → ordenes_trabajo`	1:N (opcional)	FK `ordenes_trabajo.id_plan`, admite `NULL`	Cada vencimiento del plan genera una OT preventiva; las correctivas no tienen plan.
`tecnicos → ordenes_trabajo`	1:N	FK `ordenes_trabajo.id_tecnico`	Un técnico tiene muchas OT asignadas; cada OT tiene un técnico responsable.
`ordenes_trabajo ↔ repuestos`	N:M	Tabla `ot_repuestos`	Una OT consume varios repuestos; un repuesto se usa en muchas OT.

Diagrama entidad-relación

El diagrama usa la notación de «pata de gallo»: la doble barra indica «exactamente uno», la barra con círculo «cero o uno» y la pata de gallo con círculo «cero o muchos». Las relaciones de `fallas` y de `planes_preventivos` con `ordenes_trabajo` son opcionales: una falla puede estar pendiente, sin orden, y una orden correctiva no tiene plan. Las filas sombreadas son las claves primarias; `UK` marca los campos únicos (códigos, legajo y número de orden).

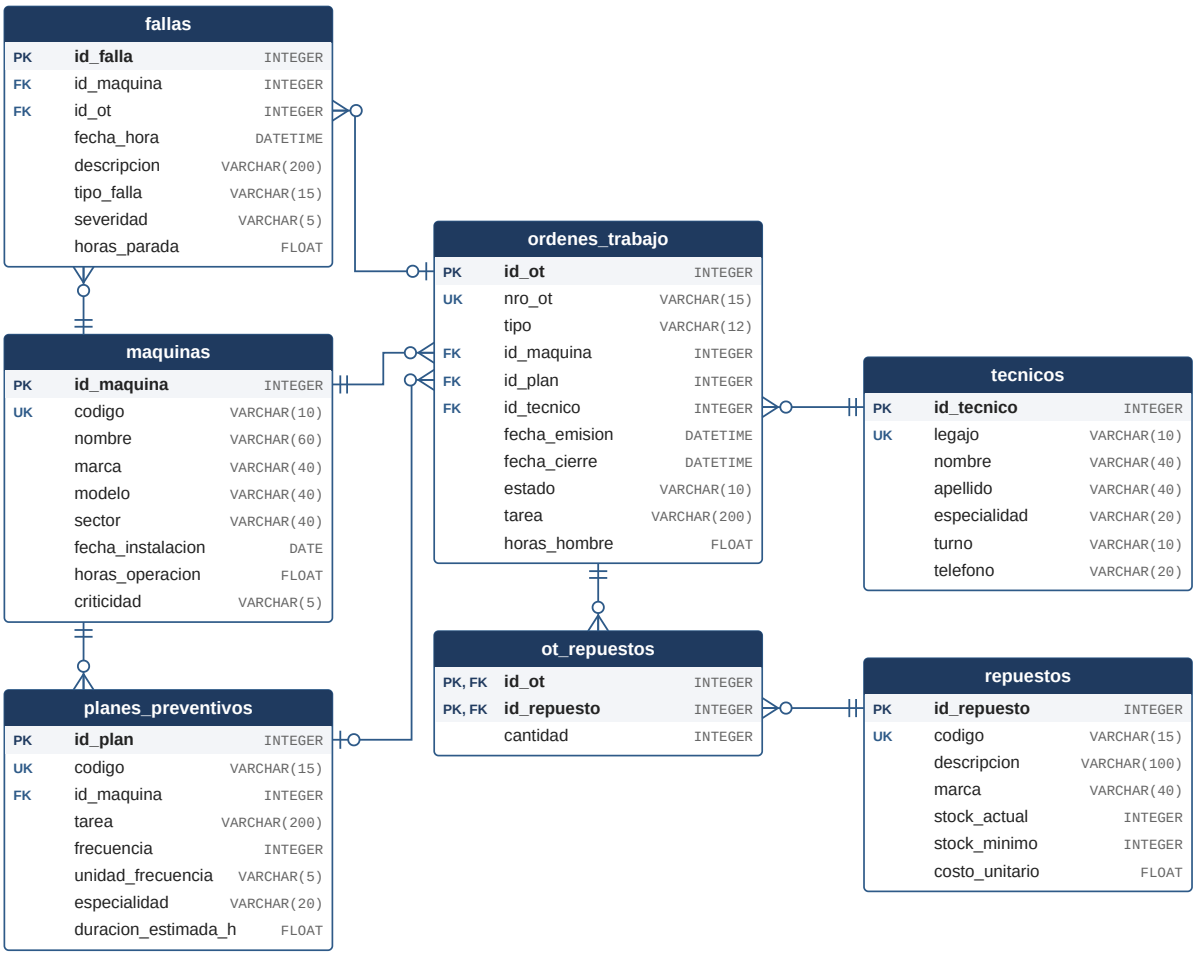


Figura 7. Diagrama entidad-relación del modelo GMAO, con claves primarias (PK), claves foráneas (FK), campos únicos (UK) y cardinalidades

Reglas de integridad del proceso

El origen de una orden depende de su tipo. Estas reglas se hacen cumplir en la propia base, para que no dependan del cuidado de quien carga los datos:

Tabla 13. Reglas de integridad del proceso

Regla	Cómo se hace cumplir
Una OT preventiva tiene plan; una correctiva no.	`CHECK` en `ordenes_trabajo`.
Una OT preventiva es de la misma máquina que su plan.	*Trigger* sobre `ordenes_trabajo`.
Una falla solo se asigna a una OT **correctiva** de la **misma máquina**.	*Trigger* sobre `fallas`.

Un `CHECK` solo puede comparar columnas de la misma fila. Las dos últimas reglas necesitan consultar otra tabla, y para eso SQLite ofrece los *triggers*: sentencias que se ejecutan automáticamente antes de una inserción o modificación y que pueden cancelarla con un mensaje de error. Se implementan en el Paso 5.

La orden de trabajo guarda `id_maquina` aunque la máquina también podría deducirse de su falla o de su plan. Es una redundancia controlada: permite listar las órdenes con su máquina mediante un vínculo directo, y su coherencia queda garantizada por las reglas anteriores.

Fase III — Implementación y carga de datos mediante SQL (La Construcción)

En esta fase el modelo de la Fase II se convierte en una base de datos real. Primero se escriben las sentencias de creación de tablas, con todas sus restricciones de integridad (Paso 5), y después se cargan datos simulados de planta (Paso 6). Los dos scripts se ejecutaron en DB Browser for SQLite sobre una base vacía, en ese orden, y se verificó que la base acepta los datos válidos y rechaza los inválidos.

Paso 5 — Scripts de creación de tablas (DDL)

Justificación teórica

SQL se divide en sublenguajes según su función. El **DDL** (*Data Definition Language*, lenguaje de definición de datos) crea, modifica y elimina la estructura de la base: tablas, índices y *triggers*. Su sentencia principal es `CREATE TABLE`, que define el nombre de la tabla, sus columnas con el tipo de dato y las **restricciones de integridad** que todo registro debe cumplir:

Tabla 14. Justificación teórica

Restricción	Qué garantiza	Ejemplo en el modelo
`PRIMARY KEY`	Cada fila se identifica de forma única y su clave no puede quedar vacía.	`id_maquina INTEGER PRIMARY KEY`
`FOREIGN KEY … REFERENCES`	El valor existe en la tabla referenciada (integridad referencial).	`FOREIGN KEY (id_maquina) REFERENCES maquinas (id_maquina)`
`NOT NULL`	El campo es obligatorio.	`nombre VARCHAR(60) NOT NULL`
`UNIQUE`	No se repite el valor en toda la tabla.	`codigo VARCHAR(10) NOT NULL UNIQUE`
`CHECK`	El valor cumple una condición.	`CHECK (estado IN ('PENDIENTE', 'EN CURSO', 'CERRADA'))`

Un `CHECK` solo puede evaluar columnas de la misma fila. Las reglas que necesitan consultar otra tabla, como que una falla se asigne solo a una orden correctiva de la misma máquina, se implementan con **triggers**: bloques que SQLite ejecuta automáticamente antes de una inserción o una modificación y que, si la regla no se cumple, cancelan la operación con `RAISE(ABORT, 'mensaje')`.

En SQLite la verificación de claves foráneas viene **desactivada por defecto** y se activa en cada conexión con `PRAGMA foreign_keys = ON`. Por eso el script empieza con esa sentencia. A continuación borra las tablas si ya existían (`DROP TABLE IF EXISTS`), de modo que se puede volver a ejecutar desde cero. El borrado se hace en orden inverso a las dependencias: primero las tablas que referencian a otras.

Las tablas se crean en el orden contrario, empezando por las que no dependen de ninguna: `maquinas`, `tecnicos` y `repuestos`; después `planes_preventivos`, `ordenes_trabajo` y `fallas`; y al final la tabla intermedia `ot_repuestos`.

Procedimiento

1. En DB Browser for SQLite se eligió **Nueva base de datos** y se guardó el archivo `gmao.db`. El asistente de creación de tablas se cerró sin usarlo, porque las tablas se crean por código.
2. En **Editar → Preferencias → Base de datos** se verificó que la opción de claves foráneas estuviera activada. De todas formas, el script incluye el `PRAGMA`.
3. En la pestaña **Ejecutar SQL** se cargó el script `paso5-ddl.sql` y se ejecutó completo con **Ejecutar todo** (`F5`).
4. Se revisó el panel de mensajes: la ejecución terminó sin errores.
5. Se confirmaron los cambios con **Guardar cambios** (`Ctrl+S`).
6. En la pestaña **Estructura** se comprobaron las 7 tablas, sus columnas y los 4 *triggers*.

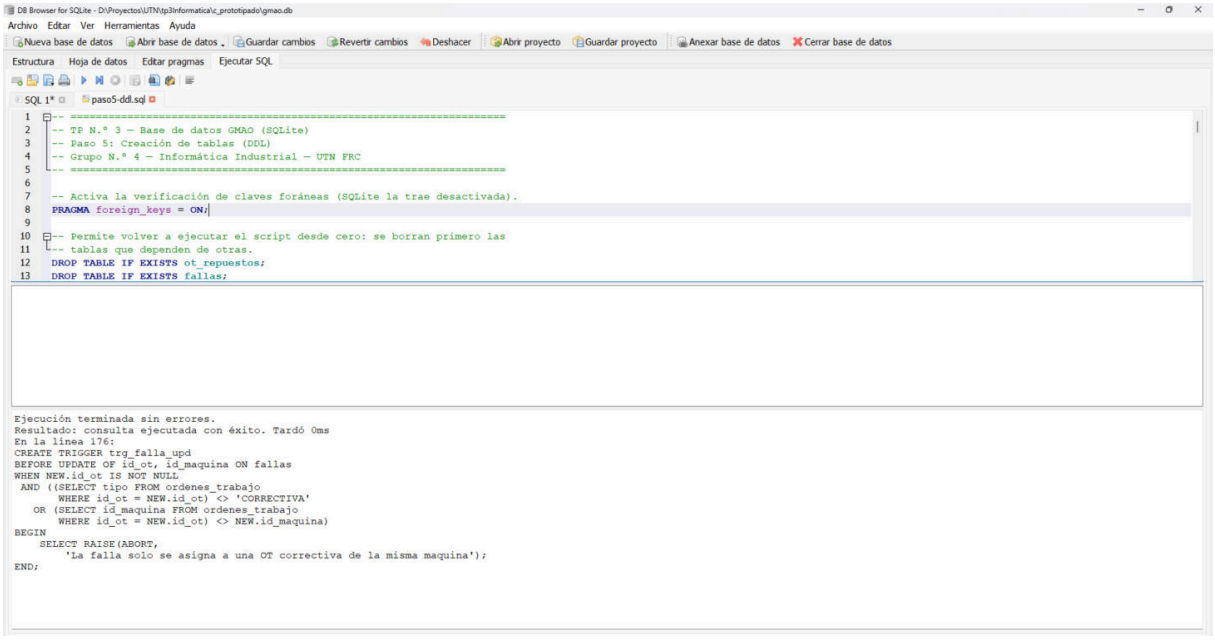


Figura 8. Ejecución de paso5-ddl.sql en la pestaña Ejecutar SQL, terminada sin errores

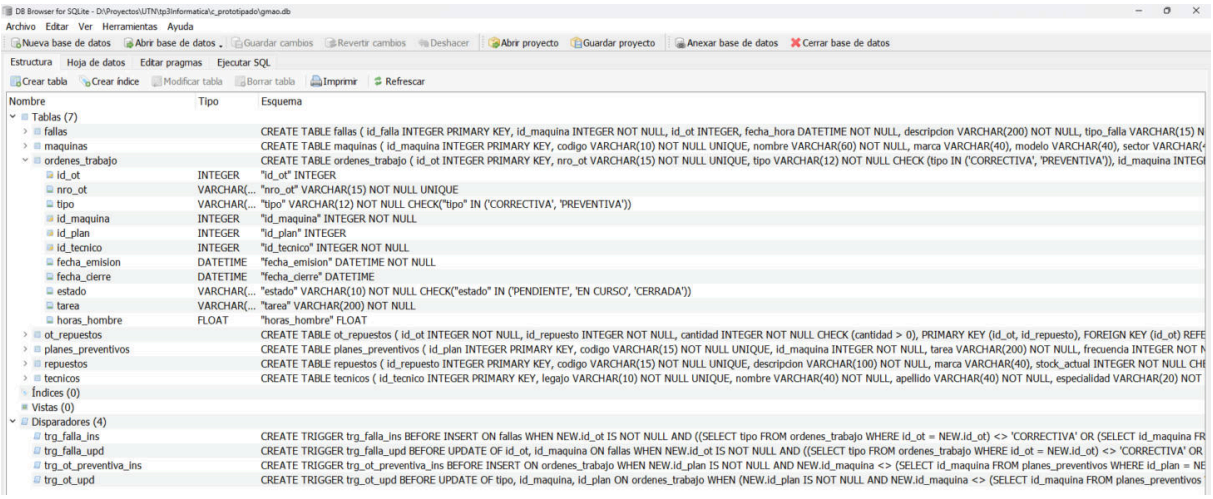


Figura 9. Pestaña Estructura: las 7 tablas (con las columnas de ordenes_trabajo desplegadas) y los 4 disparadores

Activación de claves foráneas y reinicio de la base

```
-- ================================================================
-- TP N.º 3 — Base de datos GMAO (SQLite)
-- Paso 5: Creación de tablas (DDL)
-- Grupo N.º 4 — Informática Industrial — UTN FRC
-- ================================================================

-- Activa la verificación de claves foráneas (SQLite la trae desactivada).
PRAGMA foreign_keys = ON;

-- Permite volver a ejecutar el script desde cero: se borran primero las
-- tablas que dependen de otras.
DROP TABLE IF EXISTS ot_repuestos;
DROP TABLE IF EXISTS fallas;
DROP TABLE IF EXISTS ordenes_trabajo;
DROP TABLE IF EXISTS planes_preventivos;
DROP TABLE IF EXISTS repuestos;
DROP TABLE IF EXISTS tecnicos;
DROP TABLE IF EXISTS maquinas;
```

Tabla `maquinas`

```
-- ----------------------------------------------------------------
-- Activos de la planta
-- ----------------------------------------------------------------
CREATE TABLE maquinas (
    id_maquina        INTEGER      PRIMARY KEY,
    codigo            VARCHAR(10)  NOT NULL UNIQUE,
    nombre            VARCHAR(60)  NOT NULL,
    marca             VARCHAR(40),
    modelo            VARCHAR(40),
    sector            VARCHAR(40)  NOT NULL,
    fecha_instalacion DATE,
    horas_operacion   FLOAT,
    criticidad        VARCHAR(5)   NOT NULL
                      CHECK (criticidad IN ('ALTA', 'MEDIA', 'BAJA'))
);
```

Tabla `tecnicos`

```
-- ---------------------------------------------------------------------
-- Personal de mantenimiento
-- ---------------------------------------------------------------------
CREATE TABLE tecnicos (
    id_tecnico   INTEGER      PRIMARY KEY,
    legajo       VARCHAR(10)  NOT NULL UNIQUE,
    nombre       VARCHAR(40)  NOT NULL,
    apellido     VARCHAR(40)  NOT NULL,
    especialidad VARCHAR(20)  NOT NULL,
    turno        VARCHAR(10)  NOT NULL
                 CHECK (turno IN ('MAÑANA', 'TARDE', 'NOCHE')),
    telefono     VARCHAR(20)
);
```

Tabla `repuestos`

Además de lo definido en el Paso 3, el stock actual y el mínimo no pueden ser negativos.

```
-- ---------------------------------------------------------------------
-- Catálogo y stock de repuestos
-- ---------------------------------------------------------------------
CREATE TABLE repuestos (
    id_repuesto    INTEGER       PRIMARY KEY,
    codigo         VARCHAR(15)   NOT NULL UNIQUE,
    descripcion    VARCHAR(100)  NOT NULL,
    marca          VARCHAR(40),
    stock_actual   INTEGER       NOT NULL CHECK (stock_actual >= 0),
    stock_minimo   INTEGER       NOT NULL CHECK (stock_minimo >= 0),
    costo_unitario FLOAT
);
```

Tabla `planes_preventivos`

```
-- ---------------------------------------------------------------------
-- Planes de mantenimiento preventivo (origen de las OT preventivas)
-- ---------------------------------------------------------------------
CREATE TABLE planes_preventivos (
    id_plan             INTEGER       PRIMARY KEY,
    codigo              VARCHAR(15)   NOT NULL UNIQUE,
    id_maquina          INTEGER       NOT NULL,
    tarea               VARCHAR(200)  NOT NULL,
    frecuencia          INTEGER       NOT NULL CHECK (frecuencia > 0),
    unidad_frecuencia   VARCHAR(5)    NOT NULL
                        CHECK (unidad_frecuencia IN ('DIAS', 'HORAS')),
    especialidad        VARCHAR(20)   NOT NULL,
    duracion_estimada_h FLOAT,
    FOREIGN KEY (id_maquina) REFERENCES maquinas (id_maquina)
);
```

Tabla `ordenes_trabajo`

El `CHECK` de tabla, al final de las columnas, exige que el origen de la orden corresponda a su tipo: la preventiva tiene plan y la correctiva no.

```
-- ---------------------------------------------------------------------
-- Órdenes de trabajo (correctivas y preventivas)
-- ---------------------------------------------------------------------
```

```
CREATE TABLE ordenes_trabajo (
    id_ot         INTEGER       PRIMARY KEY,
    nro_ot        VARCHAR(15)   NOT NULL UNIQUE,
    tipo          VARCHAR(12)   NOT NULL
                  CHECK (tipo IN ('CORRECTIVA', 'PREVENTIVA')),
    id_maquina    INTEGER       NOT NULL,
    id_plan       INTEGER,
    id_tecnico    INTEGER       NOT NULL,
    fecha_emision DATETIME      NOT NULL,
    fecha_cierre  DATETIME,
    estado        VARCHAR(10)   NOT NULL
                  CHECK (estado IN ('PENDIENTE', 'EN CURSO', 'CERRADA')),
    tarea         VARCHAR(200)  NOT NULL,
    horas_hombre  FLOAT,
    -- La preventiva nace de un plan; la correctiva no tiene plan.
    CHECK ((tipo = 'PREVENTIVA' AND id_plan IS NOT NULL)
        OR (tipo = 'CORRECTIVA' AND id_plan IS NULL)),
    FOREIGN KEY (id_maquina) REFERENCES maquinas (id_maquina),
    FOREIGN KEY (id_plan)    REFERENCES planes_preventivos (id_plan),
    FOREIGN KEY (id_tecnico) REFERENCES tecnicos (id_tecnico)
);
```

Tabla `fallas`

`id_ot` admite `NULL`: una falla puede registrarse antes de que se emita su orden de trabajo, y queda pendiente.

```
-- ---------------------------------------------------------------------
-- Historial de fallas (origen de las OT correctivas)
-- ---------------------------------------------------------------------
CREATE TABLE fallas (
    id_falla     INTEGER       PRIMARY KEY,
    id_maquina   INTEGER       NOT NULL,
    id_ot        INTEGER,
    fecha_hora   DATETIME      NOT NULL,
    descripcion  VARCHAR(200)  NOT NULL,
    tipo_falla   VARCHAR(15)   NOT NULL,
    severidad    VARCHAR(5)    NOT NULL
                 CHECK (severidad IN ('ALTA', 'MEDIA', 'BAJA')),
    horas_parada FLOAT,
    FOREIGN KEY (id_maquina) REFERENCES maquinas (id_maquina),
    FOREIGN KEY (id_ot)      REFERENCES ordenes_trabajo (id_ot)
);
```

Tabla `ot_repuestos`

La clave primaria es compuesta y cada una de sus columnas es, además, clave foránea.

```
-- ---------------------------------------------------------------------
-- Repuestos consumidos por orden de trabajo (relación N:M)
-- ---------------------------------------------------------------------
CREATE TABLE ot_repuestos (
    id_ot       INTEGER  NOT NULL,
    id_repuesto INTEGER  NOT NULL,
    cantidad    INTEGER  NOT NULL CHECK (cantidad > 0),
    PRIMARY KEY (id_ot, id_repuesto),
    FOREIGN KEY (id_ot)       REFERENCES ordenes_trabajo (id_ot),
    FOREIGN KEY (id_repuesto) REFERENCES repuestos (id_repuesto)
);
```

Reglas entre tablas (triggers)

Los cuatro *triggers* hacen cumplir las reglas del Paso 4 que un CHECK no puede verificar. Se controlan tanto las inserciones como las modificaciones, para que no se pueda dejar la base en un estado incoherente con un UPDATE posterior.

```
-- ---------------------------------------------------------------------
-- Reglas entre tablas (un CHECK no puede consultar otra tabla)
-- ---------------------------------------------------------------------

-- Una OT preventiva tiene que ser de la misma máquina que su plan.
CREATE TRIGGER trg_ot_preventiva_ins
BEFORE INSERT ON ordenes_trabajo
WHEN NEW.id_plan IS NOT NULL
 AND NEW.id_maquina <> (SELECT id_maquina FROM planes_preventivos
                        WHERE id_plan = NEW.id_plan)
BEGIN
    SELECT RAISE(ABORT,
        'La OT preventiva debe ser de la misma maquina que su plan');
END;

-- Al modificar una OT se vuelve a controlar su plan y las fallas que atiende.
CREATE TRIGGER trg_ot_upd
BEFORE UPDATE OF tipo, id_maquina, id_plan ON ordenes_trabajo
WHEN (NEW.id_plan IS NOT NULL
      AND NEW.id_maquina <> (SELECT id_maquina FROM planes_preventivos
                             WHERE id_plan = NEW.id_plan))
  OR (NEW.tipo = 'PREVENTIVA'
      AND EXISTS (SELECT 1 FROM fallas WHERE id_ot = NEW.id_ot))
  OR EXISTS (SELECT 1 FROM fallas
             WHERE id_ot = NEW.id_ot AND id_maquina <> NEW.id_maquina)
BEGIN
    SELECT RAISE(ABORT,
        'Cambio incoherente con el plan o las fallas de la OT');
END;

-- Una falla solo se asigna a una OT correctiva de la misma máquina.
CREATE TRIGGER trg_falla_ins
BEFORE INSERT ON fallas
WHEN NEW.id_ot IS NOT NULL
 AND ((SELECT tipo FROM ordenes_trabajo
       WHERE id_ot = NEW.id_ot) <> 'CORRECTIVA'
   OR (SELECT id_maquina FROM ordenes_trabajo
       WHERE id_ot = NEW.id_ot) <> NEW.id_maquina)
BEGIN
    SELECT RAISE(ABORT,
        'La falla solo se asigna a una OT correctiva de la misma maquina');
END;

CREATE TRIGGER trg_falla_upd
BEFORE UPDATE OF id_ot, id_maquina ON fallas
WHEN NEW.id_ot IS NOT NULL
 AND ((SELECT tipo FROM ordenes_trabajo
       WHERE id_ot = NEW.id_ot) <> 'CORRECTIVA'
   OR (SELECT id_maquina FROM ordenes_trabajo
       WHERE id_ot = NEW.id_ot) <> NEW.id_maquina)
BEGIN
    SELECT RAISE(ABORT,
        'La falla solo se asigna a una OT correctiva de la misma maquina');
END;
```

Paso 6 — Script de inserción de datos de planta (DML)

Justificación teórica

El **DML** (*Data Manipulation Language*, lenguaje de manipulación de datos) opera sobre el contenido de las tablas: `INSERT` agrega registros, `UPDATE` los modifica, `DELETE` los elimina y `SELECT` los consulta. La sentencia `INSERT INTO tabla (columnas) VALUES (…), (…)` permite cargar varios registros en una sola instrucción. Nombrar las columnas de forma explícita hace que el script no dependa del orden en que fueron creadas.

El orden de carga respeta las claves foráneas: no se puede insertar una orden de trabajo de una máquina que todavía no existe. Por eso primero se cargan `maquinas`, `tecnicos` y `repuestos`, después `planes_preventivos`, luego `ordenes_trabajo`, después `fallas` (que referencian a su orden) y al final `ot_repuestos`.

Los valores ausentes se cargan como `NULL`: la fecha de cierre de una orden abierta, el plan de una orden correctiva o la orden de una falla pendiente. Las fechas se escriben en formato ISO 8601. Los cambios quedan en una transacción hasta que se confirman con **Guardar cambios**: si alguna sentencia falla, DB Browser no guarda nada a medias.

Procedimiento

1. Sobre la base `gmao.db` creada en el Paso 5 se abrió la pestaña **Ejecutar SQL**.
2. Se cargó el script `paso6-dml.sql` y se ejecutó completo (`F5`).
3. Se revisó el panel de mensajes: todas las inserciones terminaron sin errores.
4. Se confirmaron los cambios con **Guardar cambios** (`Ctrl+S`).
5. En la pestaña **Hoja de datos** se revisó cada tabla.

Datos cargados

Los datos son **simulados**, pero se eligieron para que representen una planta real y para que las consultas de la Fase IV devuelvan resultados significativos:

Tabla 15. Datos cargados

Tabla	Registros	Qué permite analizar
`maquinas`	6	Equipos reales de distintos sectores y criticidades (Haas, Atlas Copco, ABB, Engel).
`tecnicos`	5	Especialidades y turnos distintos.
`repuestos`	7	Dos repuestos por debajo del stock mínimo (filtro del compresor y placa del robot).
`planes_preventivos`	6	Planes por calendario (días) y por horómetro (horas).
`ordenes_trabajo`	13	8 correctivas y 5 preventivas, en los tres estados.
`fallas`	10	Fallas entre septiembre y octubre de 2026; el torno CNC es el activo con más fallas; dos fallas se atienden en una misma orden y una queda pendiente.
`ot_repuestos`	6	Consumo de repuestos en correctivas y preventivas.

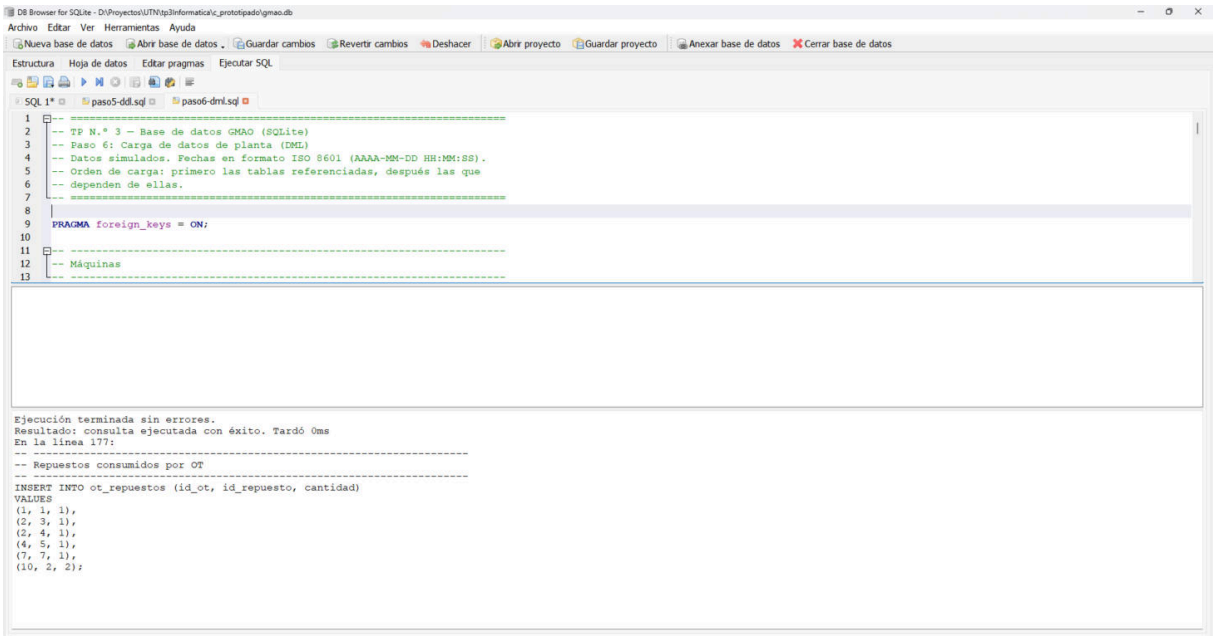
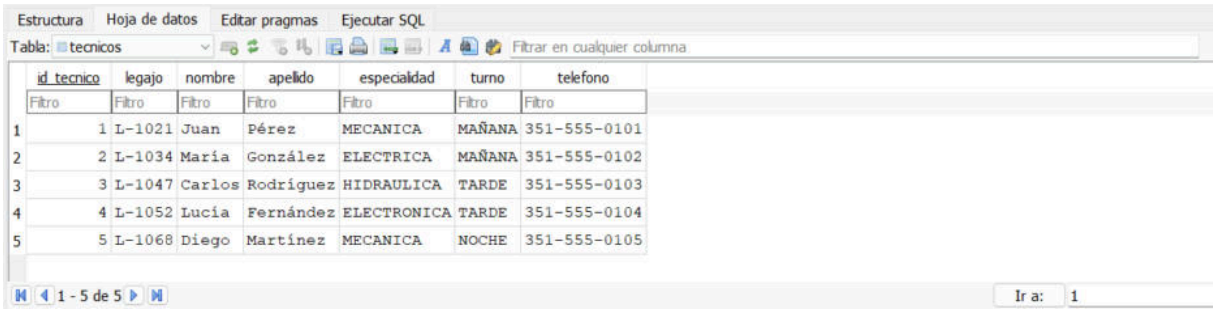


Figura 10. Ejecución de paso6-dml.sql en la pestaña Ejecutar SQL, terminada sin errores



Figura 11. Pestaña Hoja de datos: tabla maquinas (6 registros)



Estructura | Hoja de datos | Editar pragmas | Ejecutar SQL

Tabla: tecnicos — Filtrar en cualquier columna

	id_tecnico	legajo	nombre	apellido	especialidad	turno	telefono
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	L-1021	Juan	Pérez	MECANICA	MAÑANA	351-555-0101
2	2	L-1034	María	González	ELECTRICA	MAÑANA	351-555-0102
3	3	L-1047	Carlos	Rodríguez	HIDRAULICA	TARDE	351-555-0103
4	4	L-1052	Lucía	Fernández	ELECTRONICA	TARDE	351-555-0104
5	5	L-1068	Diego	Martínez	MECANICA	NOCHE	351-555-0105

1 - 5 de 5 — Ir a: 1

Figura 12. Pestaña Hoja de datos: tabla tecnicos (5 registros)



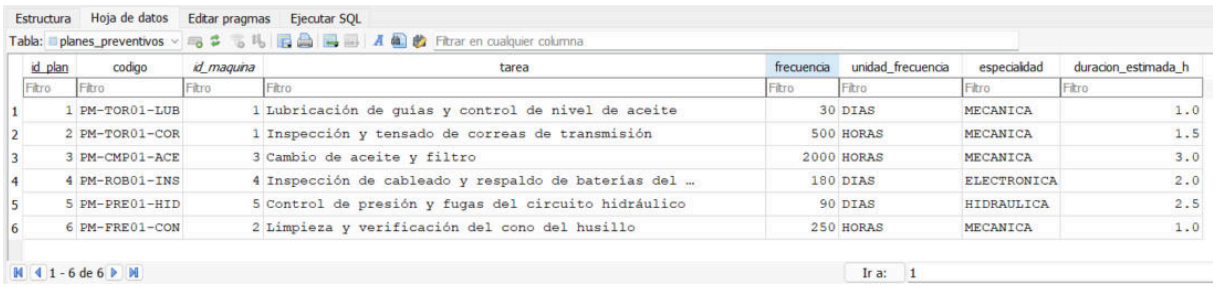
Estructura | Hoja de datos | Editar pragmas | Ejecutar SQL

Tabla: repuestos — Filtrar en cualquier columna

	id_repuesto	codigo	descripcion	marca	stock_actual	stock_minimo	costo_unitario
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	COR-5V710	Correa de transmisión 5V710	Gates	2	2	38500.0
2	2	ROD-6206-2RS	Rodamiento rígido de bolas 6206-2RS	SKF	12	6	9800.0
3	3	FIL-AC-GA37	Filtro de aceite para compresor GA 37	Atlas Copco	1	3	52000.0
4	4	ACE-RIF-20L	Aceite para compresor de tornillo, 20 L	Atlas Copco	4	2	310000.0
5	5	PLA-SMB-IRB	Placa de medición serie (SMB) del robot	ABB	0	1	1850000.0
6	6	RES-800W	Resistencia calefactora de cilindro 800 W	Engel	3	2	74000.0
7	7	SEL-CIL-80	Kit de sellos para cilindro hidráulico 80	Parker	2	2	126000.0

1 - 7 de 7 — Ir a: 1

Figura 13. Pestaña Hoja de datos: tabla repuestos (7 registros)



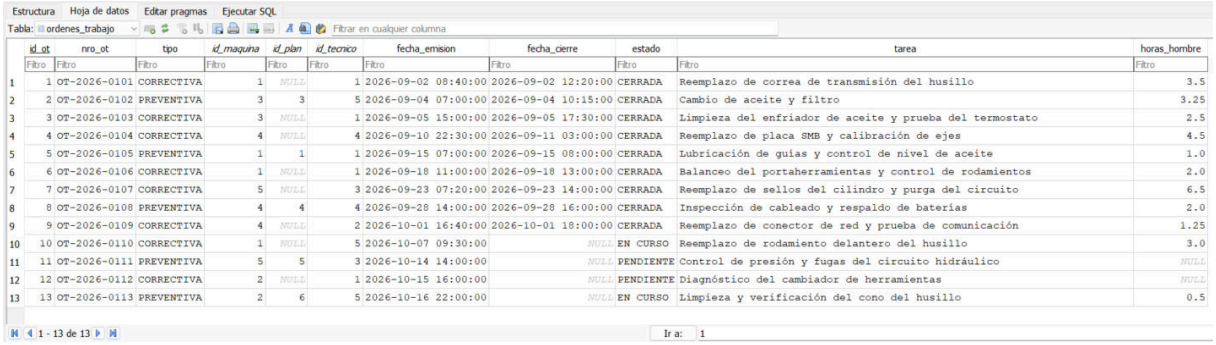
Estructura | Hoja de datos | Editar pragmas | Ejecutar SQL

Tabla: planes_preventivos — Filtrar en cualquier columna

	id_plan	codigo	id_maquina	tarea	frecuencia	unidad_frecuencia	especialidad	duracion_estimada_h
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	PM-TOR01-LUB	1	Lubricación de guías y control de nivel de aceite	30	DIAS	MECANICA	1.0
2	2	PM-TOR01-COR	1	Inspección y tensado de correas de transmisión	500	HORAS	MECANICA	1.5
3	3	PM-CMP01-ACE	3	Cambio de aceite y filtro	2000	HORAS	MECANICA	3.0
4	4	PM-ROB01-INS	4	Inspección de cableado y respaldo de baterías del …	180	DIAS	ELECTRONICA	2.0
5	5	PM-PRE01-HID	5	Control de presión y fugas del circuito hidráulico	90	DIAS	HIDRAULICA	2.5
6	6	PM-FRE01-CON	2	Limpieza y verificación del cono del husillo	250	HORAS	MECANICA	1.0

1 - 6 de 6 — Ir a: 1

Figura 14. Pestaña Hoja de datos: tabla planes_preventivos (6 registros)



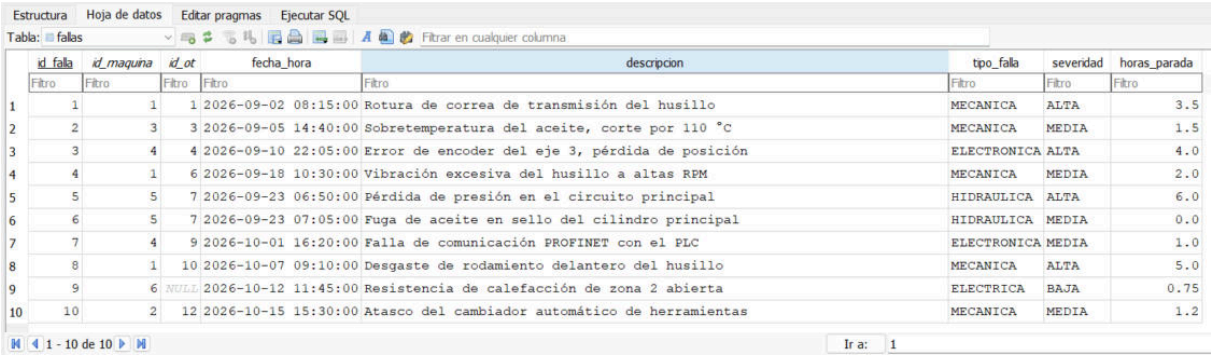
Estructura | Hoja de datos | Editar pragmas | Ejecutar SQL

Tabla: ordenes_trabajo — Filtrar en cualquier columna

	id_ot	nro_ot	tipo	id_maquina	id_plan	id_tecnico	fecha_emision	fecha_cierre	estado	tarea	horas_hombre
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	OT-2026-0101	CORRECTIVA	1	NULL	1	2026-09-02 08:40:00	2026-09-02 12:20:00	CERRADA	Reemplazo de correa de transmisión del husillo	3.5
2	2	OT-2026-0102	PREVENTIVA	3	3	5	2026-09-04 07:00:00	2026-09-04 10:15:00	CERRADA	Cambio de aceite y filtro	3.25
3	3	OT-2026-0103	CORRECTIVA	3	NULL	1	2026-09-05 15:00:00	2026-09-05 17:30:00	CERRADA	Limpieza del enfriador de aceite y prueba del termostato	2.5
4	4	OT-2026-0104	CORRECTIVA	4	NULL	4	2026-09-10 22:30:00	2026-09-11 03:00:00	CERRADA	Reemplazo de placa SMB y calibración de ejes	4.5
5	5	OT-2026-0105	PREVENTIVA	1	1	1	2026-09-15 07:00:00	2026-09-15 08:00:00	CERRADA	Lubricación de guías y control de nivel de aceite	1.0
6	6	OT-2026-0106	CORRECTIVA	1	NULL	1	2026-09-18 11:00:00	2026-09-18 13:00:00	CERRADA	Balanceo del portaherramientas y control de rodamientos	2.0
7	7	OT-2026-0107	CORRECTIVA	5	NULL	3	2026-09-23 07:20:00	2026-09-23 14:00:00	CERRADA	Reemplazo de sellos del cilindro y purga del circuito	6.5
8	8	OT-2026-0108	PREVENTIVA	4	4	4	2026-09-28 14:00:00	2026-09-28 16:00:00	CERRADA	Inspección de cableado y respaldo de baterías	2.0
9	9	OT-2026-0109	CORRECTIVA	4	NULL	2	2026-10-01 16:40:00	2026-10-01 18:00:00	CERRADA	Reemplazo de conector de red y prueba de comunicación	1.25
10	10	OT-2026-0110	CORRECTIVA	1	NULL	5	2026-10-07 09:30:00	NULL	EN CURSO	Reemplazo de rodamiento delantero del husillo	3.0
11	11	OT-2026-0111	PREVENTIVA	5	5	3	2026-10-14 14:00:00	NULL	PENDIENTE	Control de presión y fugas del circuito hidráulico	NULL
12	12	OT-2026-0112	CORRECTIVA	2	NULL	1	2026-10-15 16:00:00	NULL	PENDIENTE	Diagnóstico del cambiador de herramientas	NULL
13	13	OT-2026-0113	PREVENTIVA	2	6	5	2026-10-16 22:00:00	NULL	EN CURSO	Limpieza y verificación del cono del husillo	0.5

1 - 13 de 13 — Ir a: 1

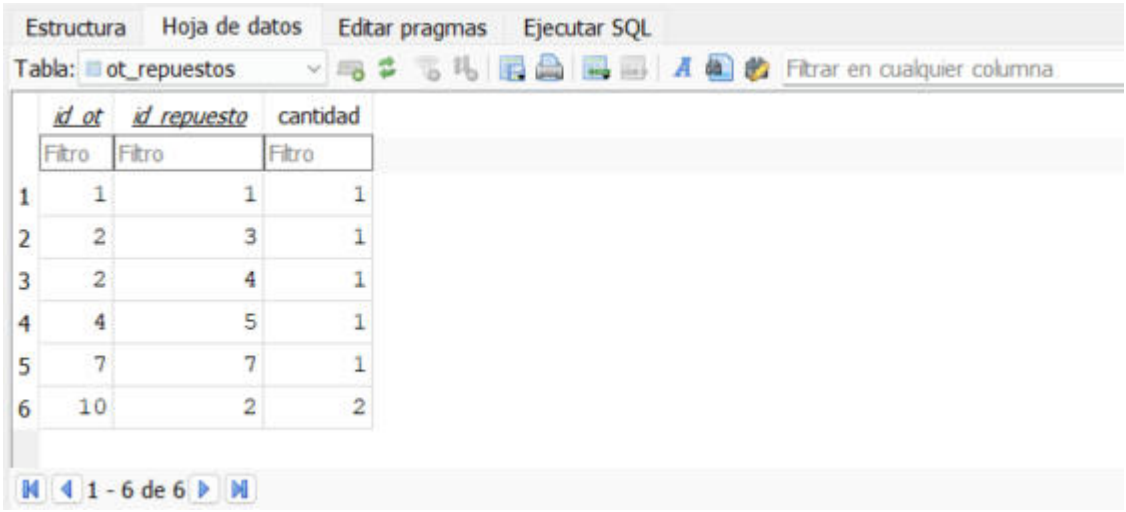
Figura 15. Pestaña Hoja de datos: tabla ordenes_trabajo (13 registros)


Estructura Hoja de datos Editar pragmas Ejecutar SQL
Tabla: fallas Filtrar en cualquier columna

	id_falla	id_maquina	id_ot	fecha_hora	descripcion	tipo_falla	severidad	horas_parada
	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	1	1	2026-09-02 08:15:00	Rotura de correa de transmisión del husillo	MECANICA	ALTA	3.5
2	2	3	3	2026-09-05 14:40:00	Sobretemperatura del aceite, corte por 110 °C	MECANICA	MEDIA	1.5
3	3	4	4	2026-09-10 22:05:00	Error de encoder del eje 3, pérdida de posición	ELECTRONICA	ALTA	4.0
4	4	1	6	2026-09-18 10:30:00	Vibración excesiva del husillo a altas RPM	MECANICA	MEDIA	2.0
5	5	5	7	2026-09-23 06:50:00	Pérdida de presión en el circuito principal	HIDRAULICA	ALTA	6.0
6	6	5	7	2026-09-23 07:05:00	Fuga de aceite en sello del cilindro principal	HIDRAULICA	MEDIA	0.0
7	7	4	9	2026-10-01 16:20:00	Falla de comunicación PROFINET con el PLC	ELECTRONICA	MEDIA	1.0
8	8	1	10	2026-10-07 09:10:00	Desgaste de rodamiento delantero del husillo	MECANICA	ALTA	5.0
9	9	6	NULL	2026-10-12 11:45:00	Resistencia de calefacción de zona 2 abierta	ELECTRICA	BAJA	0.75
10	10	2	12	2026-10-15 15:30:00	Atasco del cambiador automático de herramientas	MECANICA	MEDIA	1.2

1 - 10 de 10 Ir a: 1

Figura 16. Pestaña Hoja de datos: tabla fallas (10 registros)


Estructura Hoja de datos Editar pragmas Ejecutar SQL
Tabla: ot_repuestos Filtrar en cualquier columna

	id_ot	id_repuesto	cantidad
	Filtro	Filtro	Filtro
1	1	1	1
2	2	3	1
3	2	4	1
4	4	5	1
5	7	7	1
6	10	2	2

1 - 6 de 6

Figura 17. Pestaña Hoja de datos: tabla ot_repuestos (6 registros)

Activación de claves foráneas

```
-- ============================================================================
-- TP N.º 3 — Base de datos GMAO (SQLite)
-- Paso 6: Carga de datos de planta (DML)
-- Datos simulados. Fechas en formato ISO 8601 (AAAA-MM-DD HH:MM:SS).
-- Orden de carga: primero las tablas referenciadas, después las que
-- dependen de ellas.
-- ============================================================================

PRAGMA foreign_keys = ON;
```

Carga de `maquinas`

```
-- ----------------------------------------------------------------------------
-- Máquinas
-- ----------------------------------------------------------------------------
INSERT INTO maquinas (id_maquina, codigo, nombre, marca, modelo, sector,
                      fecha_instalacion, horas_operacion, criticidad)
VALUES
(1, 'TOR-01', 'Torno CNC', 'Haas', 'ST-20',
    'Mecanizado', '2021-03-15', 12850.5, 'ALTA'),
(2, 'FRE-01', 'Centro de mecanizado vertical', 'Haas', 'VF-2',
    'Mecanizado', '2020-11-02', 15420.0, 'ALTA'),
(3, 'CMP-01', 'Compresor de tornillo', 'Atlas Copco', 'GA 37',
    'Servicios', '2019-07-01', 30210.0, 'MEDIA'),
```

```
(4, 'ROB-01', 'Robot de soldadura', 'ABB', 'IRB 2600',
    'Soldadura', '2022-05-20', 9870.0, 'ALTA'),
(5, 'PRE-01', 'Prensa hidráulica', 'Schuler', 'PH 250',
    'Estampado', '2018-02-12', 41200.0, 'MEDIA'),
(6, 'INY-01', 'Inyectora de plástico', 'Engel', 'e-mac 180',
    'Inyección', '2023-01-09', 6150.0, 'BAJA');
```

Carga de `tecnicos`

```
-- ------------------------------------------------------------------
-- Técnicos
-- ------------------------------------------------------------------
INSERT INTO tecnicos (id_tecnico, legajo, nombre, apellido, especialidad,
                      turno, telefono)
VALUES
(1, 'L-1021', 'Juan',   'Pérez',     'MECANICA',    'MAÑANA', '351-555-0101'),
(2, 'L-1034', 'María',  'González',  'ELECTRICA',   'MAÑANA', '351-555-0102'),
(3, 'L-1047', 'Carlos', 'Rodríguez', 'HIDRAULICA',  'TARDE',  '351-555-0103'),
(4, 'L-1052', 'Lucía',  'Fernández', 'ELECTRONICA', 'TARDE',  '351-555-0104'),
(5, 'L-1068', 'Diego',  'Martínez',  'MECANICA',    'NOCHE',  '351-555-0105');
```

Carga de `repuestos`

```
-- ------------------------------------------------------------------
-- Repuestos (stock actual y punto de reposición)
-- ------------------------------------------------------------------
INSERT INTO repuestos (id_repuesto, codigo, descripcion, marca,
                       stock_actual, stock_minimo, costo_unitario)
VALUES
(1, 'COR-5V710', 'Correa de transmisión 5V710', 'Gates',
    2, 2, 38500.0),
(2, 'ROD-6206-2RS', 'Rodamiento rígido de bolas 6206-2RS', 'SKF',
    12, 6, 9800.0),
(3, 'FIL-AC-GA37', 'Filtro de aceite para compresor GA 37', 'Atlas Copco',
    1, 3, 52000.0),
(4, 'ACE-RIF-20L', 'Aceite para compresor de tornillo, 20 L', 'Atlas Copco',
    4, 2, 310000.0),
(5, 'PLA-SMB-IRB', 'Placa de medición serie (SMB) del robot', 'ABB',
    0, 1, 1850000.0),
(6, 'RES-800W', 'Resistencia calefactora de cilindro 800 W', 'Engel',
    3, 2, 74000.0),
(7, 'SEL-CIL-80', 'Kit de sellos para cilindro hidráulico 80', 'Parker',
    2, 2, 126000.0);
```

Carga de `planes_preventivos`

```
-- ------------------------------------------------------------------
-- Planes preventivos (por calendario: DIAS; por horómetro: HORAS)
-- ------------------------------------------------------------------
INSERT INTO planes_preventivos (id_plan, codigo, id_maquina, tarea,
                                frecuencia, unidad_frecuencia,
                                especialidad, duracion_estimada_h)
VALUES
(1, 'PM-TOR01-LUB', 1,
    'Lubricación de guías y control de nivel de aceite',
    30, 'DIAS', 'MECANICA', 1.0),
(2, 'PM-TOR01-COR', 1,
    'Inspección y tensado de correas de transmisión',
    500, 'HORAS', 'MECANICA', 1.5),
(3, 'PM-CMP01-ACE', 3,
    'Cambio de aceite y filtro',
```

```
    2000, 'HORAS', 'MECANICA', 3.0),
(4, 'PM-ROB01-INS', 4,
    'Inspección de cableado y respaldo de baterías del controlador',
    180, 'DIAS', 'ELECTRONICA', 2.0),
(5, 'PM-PRE01-HID', 5,
    'Control de presión y fugas del circuito hidráulico',
    90, 'DIAS', 'HIDRAULICA', 2.5),
(6, 'PM-FRE01-CON', 2,
    'Limpieza y verificación del cono del husillo',
    250, 'HORAS', 'MECANICA', 1.0);
```

Carga de `ordenes_trabajo`

```
-- ------------------------------------------------------------------
-- Órdenes de trabajo
-- Las correctivas no tienen plan (id_plan NULL); las preventivas sí.
-- Las abiertas no tienen fecha de cierre.
-- ------------------------------------------------------------------
INSERT INTO ordenes_trabajo (id_ot, nro_ot, tipo, id_maquina, id_plan,
                             id_tecnico, fecha_emision, fecha_cierre,
                             estado, tarea, horas_hombre)
VALUES
(1, 'OT-2026-0101', 'CORRECTIVA', 1, NULL, 1,
    '2026-09-02 08:40:00', '2026-09-02 12:20:00', 'CERRADA',
    'Reemplazo de correa de transmisión del husillo', 3.5),
(2, 'OT-2026-0102', 'PREVENTIVA', 3, 3, 5,
    '2026-09-04 07:00:00', '2026-09-04 10:15:00', 'CERRADA',
    'Cambio de aceite y filtro', 3.25),
(3, 'OT-2026-0103', 'CORRECTIVA', 3, NULL, 1,
    '2026-09-05 15:00:00', '2026-09-05 17:30:00', 'CERRADA',
    'Limpieza del enfriador de aceite y prueba del termostato', 2.5),
(4, 'OT-2026-0104', 'CORRECTIVA', 4, NULL, 4,
    '2026-09-10 22:30:00', '2026-09-11 03:00:00', 'CERRADA',
    'Reemplazo de placa SMB y calibración de ejes', 4.5),
(5, 'OT-2026-0105', 'PREVENTIVA', 1, 1, 1,
    '2026-09-15 07:00:00', '2026-09-15 08:00:00', 'CERRADA',
    'Lubricación de guías y control de nivel de aceite', 1.0),
(6, 'OT-2026-0106', 'CORRECTIVA', 1, NULL, 1,
    '2026-09-18 11:00:00', '2026-09-18 13:00:00', 'CERRADA',
    'Balanceo del portaherramientas y control de rodamientos', 2.0),
(7, 'OT-2026-0107', 'CORRECTIVA', 5, NULL, 3,
    '2026-09-23 07:20:00', '2026-09-23 14:00:00', 'CERRADA',
    'Reemplazo de sellos del cilindro y purga del circuito', 6.5),
(8, 'OT-2026-0108', 'PREVENTIVA', 4, 4, 4,
    '2026-09-28 14:00:00', '2026-09-28 16:00:00', 'CERRADA',
    'Inspección de cableado y respaldo de baterías', 2.0),
(9, 'OT-2026-0109', 'CORRECTIVA', 4, NULL, 2,
    '2026-10-01 16:40:00', '2026-10-01 18:00:00', 'CERRADA',
    'Reemplazo de conector de red y prueba de comunicación', 1.25),
(10, 'OT-2026-0110', 'CORRECTIVA', 1, NULL, 5,
    '2026-10-07 09:30:00', NULL, 'EN CURSO',
    'Reemplazo de rodamiento delantero del husillo', 3.0),
(11, 'OT-2026-0111', 'PREVENTIVA', 5, 5, 3,
    '2026-10-14 14:00:00', NULL, 'PENDIENTE',
    'Control de presión y fugas del circuito hidráulico', NULL),
(12, 'OT-2026-0112', 'CORRECTIVA', 2, NULL, 1,
    '2026-10-15 16:00:00', NULL, 'PENDIENTE',
    'Diagnóstico del cambiador de herramientas', NULL),
(13, 'OT-2026-0113', 'PREVENTIVA', 2, 6, 5,
    '2026-10-16 22:00:00', NULL, 'EN CURSO',
    'Limpieza y verificación del cono del husillo', 0.5);
```

Carga de `fallas`

```
-- ------------------------------------------------------------------
-- Fallas
-- Las fallas 5 y 6 se atienden en la misma OT (7).
-- La falla 9 todavía no tiene OT: está pendiente (id_ot NULL).
-- ------------------------------------------------------------------
INSERT INTO fallas (id_falla, id_maquina, id_ot, fecha_hora, descripcion,
                    tipo_falla, severidad, horas_parada)
VALUES
(1, 1, 1, '2026-09-02 08:15:00',
    'Rotura de correa de transmisión del husillo',
    'MECANICA', 'ALTA', 3.5),
(2, 3, 3, '2026-09-05 14:40:00',
    'Sobretemperatura del aceite, corte por 110 °C',
    'MECANICA', 'MEDIA', 1.5),
(3, 4, 4, '2026-09-10 22:05:00',
    'Error de encoder del eje 3, pérdida de posición',
    'ELECTRONICA', 'ALTA', 4.0),
(4, 1, 6, '2026-09-18 10:30:00',
    'Vibración excesiva del husillo a altas RPM',
    'MECANICA', 'MEDIA', 2.0),
(5, 5, 7, '2026-09-23 06:50:00',
    'Pérdida de presión en el circuito principal',
    'HIDRAULICA', 'ALTA', 6.0),
(6, 5, 7, '2026-09-23 07:05:00',
    'Fuga de aceite en sello del cilindro principal',
    'HIDRAULICA', 'MEDIA', 0.0),
(7, 4, 9, '2026-10-01 16:20:00',
    'Falla de comunicación PROFINET con el PLC',
    'ELECTRONICA', 'MEDIA', 1.0),
(8, 1, 10, '2026-10-07 09:10:00',
    'Desgaste de rodamiento delantero del husillo',
    'MECANICA', 'ALTA', 5.0),
(9, 6, NULL, '2026-10-12 11:45:00',
    'Resistencia de calefacción de zona 2 abierta',
    'ELECTRICA', 'BAJA', 0.75),
(10, 2, 12, '2026-10-15 15:30:00',
    'Atasco del cambiador automático de herramientas',
    'MECANICA', 'MEDIA', 1.2);
```

Carga de `ot_repuestos`

```
-- ------------------------------------------------------------------
-- Repuestos consumidos por OT
-- ------------------------------------------------------------------
INSERT INTO ot_repuestos (id_ot, id_repuesto, cantidad)
VALUES
(1, 1, 1),
(2, 3, 1),
(2, 4, 1),
(4, 5, 1),
(7, 7, 1),
(10, 2, 2);
```

Verificación de la integridad

Para comprobar que las restricciones funcionan se intentó cargar datos inválidos. Estas sentencias **no forman parte del script**: se ejecutaron aparte y la base las rechazó con el mensaje indicado.

Tabla 16. Verificación de la integridad

Intento	Mensaje de SQLite
OT preventiva sin plan	`CHECK constraint failed`
OT preventiva de otra máquina que su plan	`La OT preventiva debe ser de la misma maquina que su plan`
Falla asignada a una OT preventiva	`La falla solo se asigna a una OT correctiva de la misma maquina`
Falla de una máquina inexistente	`FOREIGN KEY constraint failed`
Número de OT repetido	`UNIQUE constraint failed: ordenes_trabajo.nro_ot`
Máquina sin nombre	`NOT NULL constraint failed: maquinas.nombre`
Borrar una máquina con historial	`FOREIGN KEY constraint failed`

Por ejemplo, la primera prueba:

```
INSERT INTO ordenes_trabajo (id_ot, nro_ot, tipo, id_maquina, id_plan, id_tecnico,
fecha_emision, estado, tarea)
VALUES (90, 'OT-2026-0190', 'PREVENTIVA', 1, NULL, 1, '2026-10-20 08:00:00',
'PENDIENTE', 'Prueba');
```

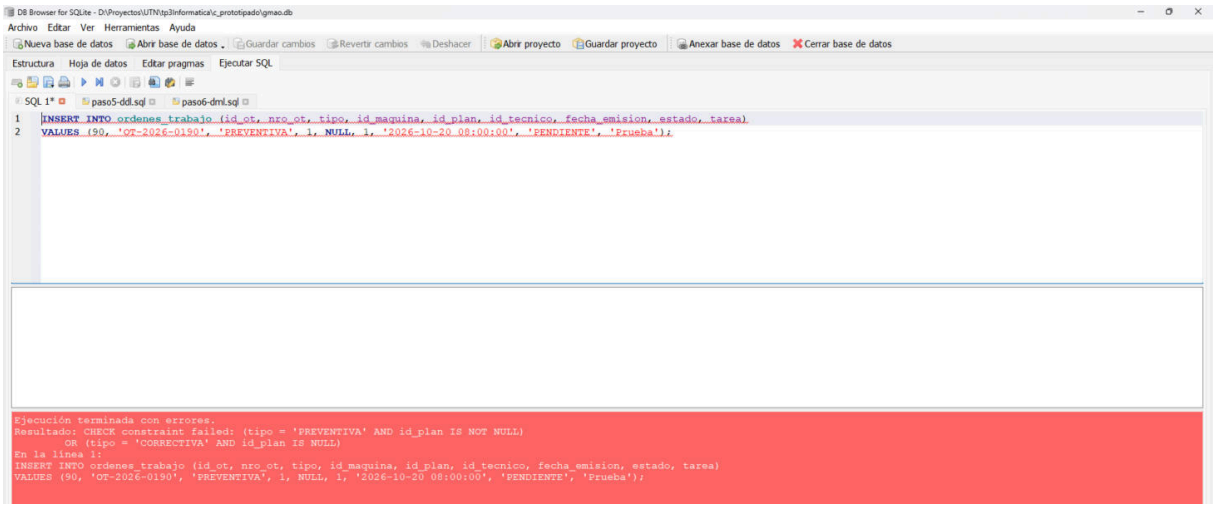


Figura 18. DB Browser rechaza la OT preventiva sin plan: CHECK constraint failed

Fase IV — Explotación de datos y reportes industriales (La Consulta)

Con la base construida y cargada, en esta fase se la consulta para responder preguntas concretas del supervisor de planta. Se escribieron siete consultas `SELECT`, en orden de complejidad creciente: filtrado sobre una sola tabla (Paso 7), estadísticas con funciones de agregación (Paso 8) y cruce de varias tablas para mostrar nombres legibles en lugar de identificadores (Paso 9). Todas se ejecutaron en DB Browser for SQLite sobre `gmao.db` y están en el script `fase4-consultas.sql`, en el mismo orden que en este informe.

Paso 7 — Consultas de selección y filtrado básico

Justificación teórica

La sentencia `SELECT` es la parte del DML que lee datos sin modificarlos. Su forma básica es:

```
SELECT columnas FROM tabla WHERE condicion ORDER BY columna [ASC | DESC];
```

- `SELECT` elige las columnas a mostrar. También acepta expresiones calculadas, a las que se les da un nombre con `AS` (alias).
- `FROM` indica la tabla de origen.
- `WHERE` filtra las filas: solo pasan las que cumplen la condición.
- `ORDER BY` ordena el resultado. Por defecto es ascendente (`ASC`); `DESC` invierte el orden.

Aunque se escribe en ese orden, el motor la evalúa en otro: primero `FROM`, después `WHERE`, luego `SELECT` y al final `ORDER BY`. Por eso, en SQL estándar, un alias definido en el `SELECT` se puede usar en el `ORDER BY` pero no en el `WHERE`. SQLite lo acepta también en el `WHERE` como extensión propia, pero no es portable a otros motores y en este trabajo no se usa.

La condición del `WHERE` se arma con operadores de comparación y operadores lógicos:

Tabla 17. Justificación teórica

Operador	Significado	Uso en las consultas
`=`, `<>`, `<`, `<=`, `>`, `>=`	Comparación	`stock_actual <= stock_minimo`
`AND`	Se cumplen las dos condiciones	Rango de fechas y severidad
`OR`	Se cumple al menos una	`severidad IN ('ALTA', 'MEDIA')` equivale a `severidad = 'ALTA' OR severidad = 'MEDIA'`
`NOT`	Niega la condición	`NOT estado = 'CERRADA'`
`BETWEEN a AND b`	Rango, con los dos extremos incluidos	Fechas de las fallas
`IN (…)`	El valor está en la lista	`severidad IN ('ALTA', 'MEDIA')`

Las fechas se guardaron como texto en formato ISO 8601 (`AAAA-MM-DD HH:MM:SS`). En ese formato el orden alfabético coincide con el cronológico, de modo que `BETWEEN` y `ORDER BY` funcionan sobre las fechas sin necesidad de convertirlas.

Procedimiento

1. Se abrió `gmao.db` en DB Browser for SQLite y se pasó a la pestaña **Ejecutar SQL**.
2. Se escribió cada consulta sola en el editor y se ejecutó con **Ejecutar todo** (`F5`). Si hay texto seleccionado, DB Browser ejecuta solo la selección.
3. Se revisó el resultado en la grilla inferior y la cantidad de filas en el panel de mensajes.
4. Como las consultas solo leen, no hubo cambios que guardar.

Consulta 7.1 — Fallas entre dos fechas

Necesidad: el supervisor revisa las fallas importantes de un período, por ejemplo para la reunión mensual de mantenimiento. Se combinan `BETWEEN` para el rango de fechas, `IN` para la severidad y `AND` para exigir las dos condiciones.

```
-- ------------------------------------------------------------------
-- Consulta 7.1 — Fallas entre dos fechas
-- Fallas de severidad ALTA o MEDIA registradas entre el 15/09 y el 10/10.
-- ------------------------------------------------------------------
SELECT id_falla, fecha_hora, id_maquina, descripcion, tipo_falla,
       severidad, horas_parada
FROM fallas
WHERE fecha_hora BETWEEN '2026-09-15 00:00:00' AND '2026-10-10 23:59:59'
  AND severidad IN ('ALTA', 'MEDIA')
ORDER BY fecha_hora;
```

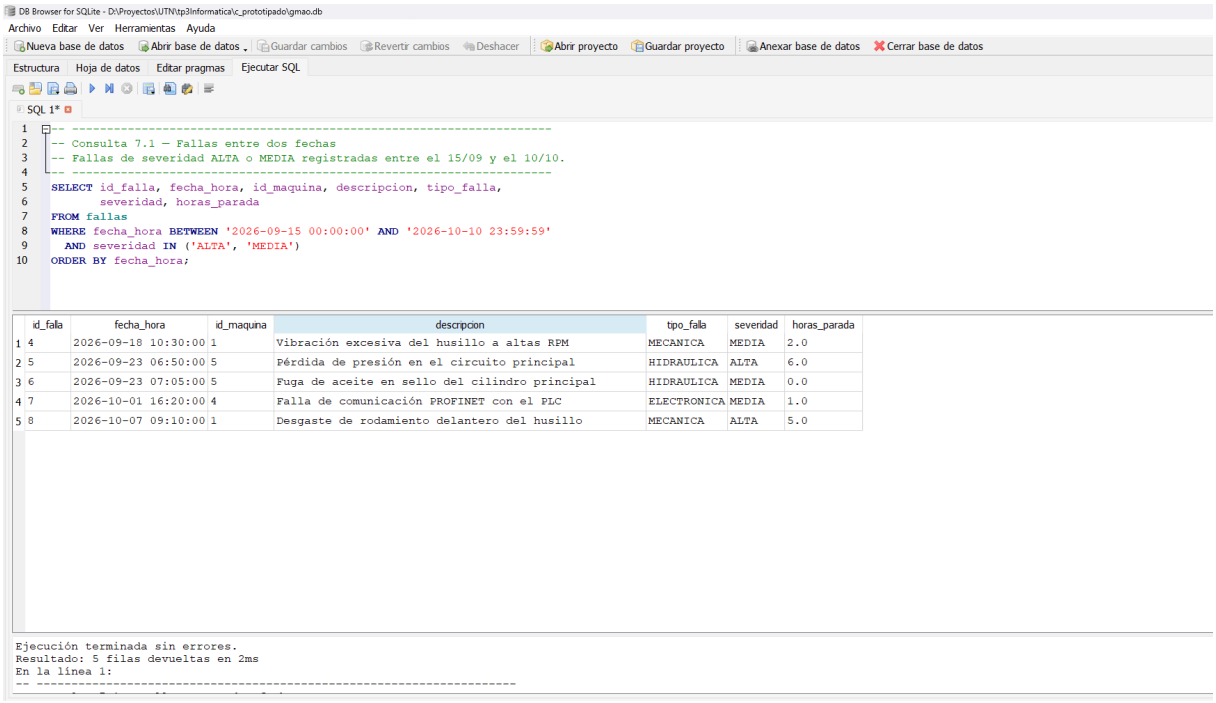


Figura 19. Consulta 7.1 en Ejecutar SQL: 5 fallas de severidad ALTA o MEDIA entre el 15/09 y el 10/10

Resultado: cinco fallas entre el 15/09 y el 10/10. Por fecha quedan afuera las fallas del 02/09, 05/09, 10/09 y 15/10; la de la inyectora (12/10) queda afuera por la fecha y por su severidad baja.

Consulta 7.2 — Repuestos con stock crítico

Necesidad: el pañol tiene que saber qué repuestos reponer antes de que una falla deje una máquina parada esperando una pieza. Un repuesto está en estado crítico cuando su stock llega al mínimo (punto de reposición). La columna calculada `faltante` ordena la lista por urgencia.

```
-- ------------------------------------------------------------------------
-- Consulta 7.2 — Repuestos con stock crítico
-- Repuestos en el punto de reposición o por debajo. Primero los que
-- más faltan.
-- ------------------------------------------------------------------------
SELECT codigo, descripcion, stock_actual, stock_minimo,
       stock_minimo - stock_actual AS faltante
FROM repuestos
WHERE stock_actual <= stock_minimo
ORDER BY faltante DESC, codigo;
```

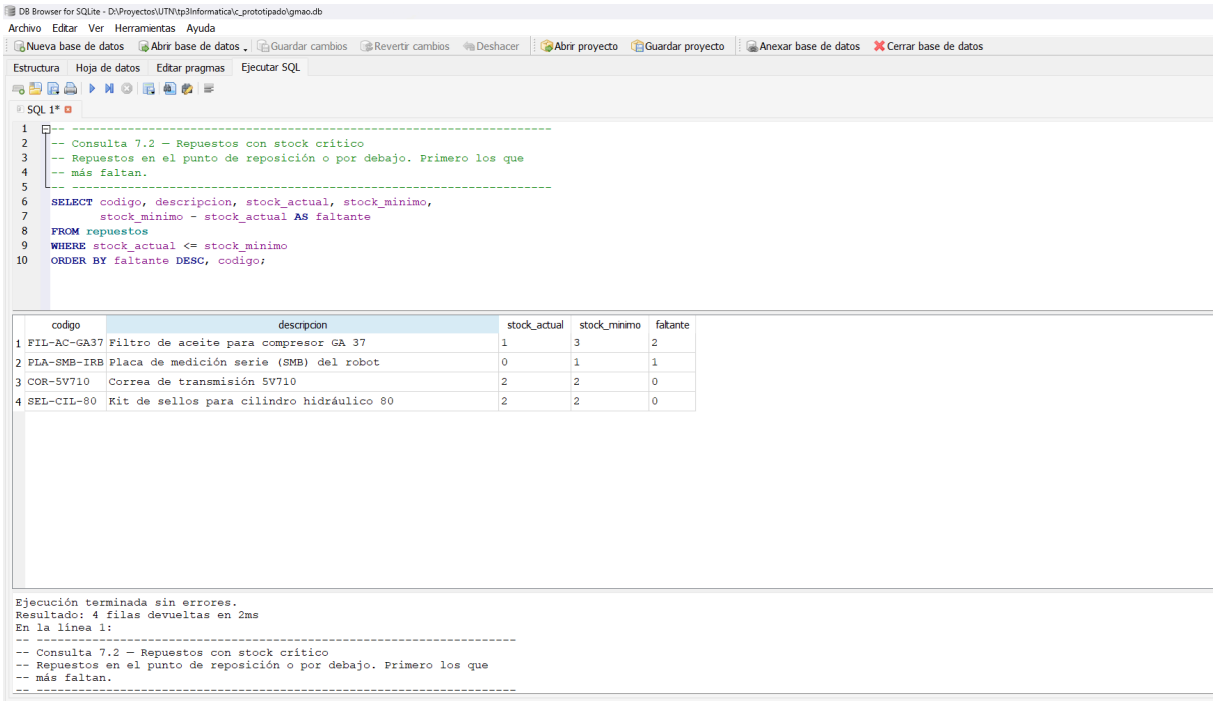


Figura 20. Consulta 7.2 en Ejecutar SQL: 4 repuestos en el stock mínimo o por debajo

Resultado: cuatro repuestos. El filtro del compresor (faltan 2) y la placa SMB del robot (stock 0) están por debajo del mínimo; la correa del torno y el kit de sellos de la prensa están justo en el mínimo.

Consulta 7.3 — Órdenes correctivas abiertas

Necesidad: las correctivas abiertas son máquinas con una falla sin resolver; el supervisor las sigue a diario. Se usa `NOT` para excluir las cerradas, lo que equivale a `estado IN ('PENDIENTE', 'EN CURSO')`.

```
-- ----------------------------------------------------------------------
-- Consulta 7.3 — Órdenes correctivas abiertas
-- Correctivas que no están cerradas, de la más antigua a la más nueva.
-- ----------------------------------------------------------------------
SELECT nro_ot, fecha_emision, id_maquina, id_tecnico, estado, tarea
FROM ordenes_trabajo
WHERE tipo = 'CORRECTIVA'
  AND NOT estado = 'CERRADA'
ORDER BY fecha_emision;
```

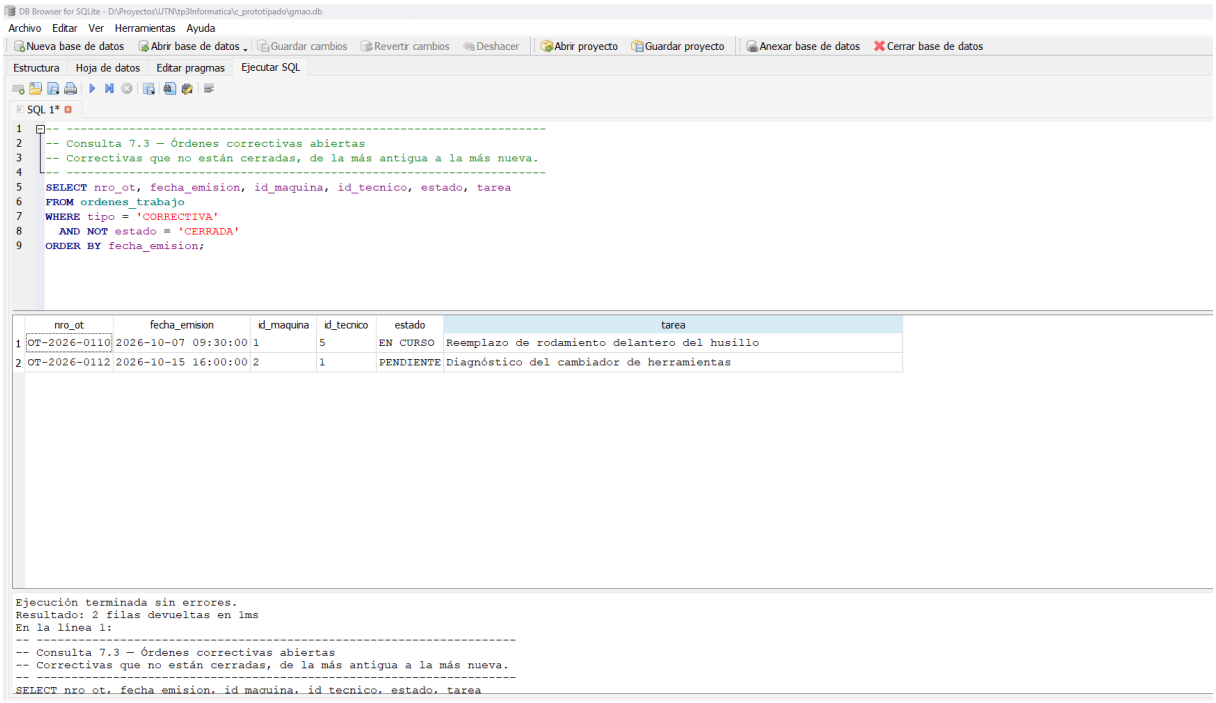


Figura 21. Consulta 7.3 en Ejecutar SQL: 2 órdenes correctivas abiertas

Resultado: dos órdenes abiertas: el cambio de rodamiento del torno CNC (en curso desde el 07/10) y el diagnóstico del cambiador de herramientas del centro de mecanizado (pendiente). Las órdenes preventivas abiertas no aparecen porque el filtro pide solo correctivas.

Paso 8 — Consultas de agrupación y estadísticas

Justificación teórica

Las **funciones de agregación** resumen un conjunto de filas en un solo valor:

Tabla 18. Justificación teórica

Función	Devuelve
`COUNT(*)`	Cantidad de filas
`SUM(columna)`	Suma de los valores
`AVG(columna)`	Promedio de los valores
`MAX(columna)` / `MIN(columna)`	Valor máximo / mínimo

Con `GROUP BY` el cálculo se hace por grupos: las filas que comparten el valor de la columna indicada forman un grupo y cada grupo da una fila del resultado. En el `SELECT` solo deberían ir las columnas del `GROUP BY` y funciones de agregación, como exige el SQL estándar. SQLite acepta otras columnas sin dar error, pero devuelve el valor de una fila cualquiera del grupo, así que el resultado no es confiable. Si hace falta filtrar grupos (por ejemplo, máquinas con más de dos fallas) se usa `HAVING`, porque `WHERE` filtra filas antes de agrupar.

Las funciones de agregación **ignoran los `NULL`**, salvo `COUNT(*)`, que cuenta filas. Esto importa en las horas-hombre: las órdenes pendientes todavía no tienen horas cargadas, cuentan como órdenes pero no entran en la suma ni en el promedio.

Procedimiento

Se siguió el mismo procedimiento que en el Paso 7: cada consulta se ejecutó sola en la pestaña **Ejecutar SQL** y se revisó la grilla de resultados. En las capturas se ensancharon las columnas de texto para que se lean completas.

Consulta 8.1 — Fallas por máquina (activo más crítico)

Necesidad: identificar el activo que más falla para priorizar su mantenimiento y revisar su plan preventivo. Se cuentan las fallas de cada máquina y se suman sus horas de parada, que son las que miden el impacto en la producción.

```
-- ------------------------------------------------------------------------
-- Consulta 8.1 — Fallas por máquina (activo más crítico)
-- Cantidad de fallas y horas de parada acumuladas por máquina.
-- ------------------------------------------------------------------------
SELECT id_maquina,
       COUNT(*)          AS cantidad_fallas,
       SUM(horas_parada) AS horas_parada_total,
       MAX(fecha_hora)   AS ultima_falla
FROM fallas
GROUP BY id_maquina
ORDER BY cantidad_fallas DESC, horas_parada_total DESC;
```

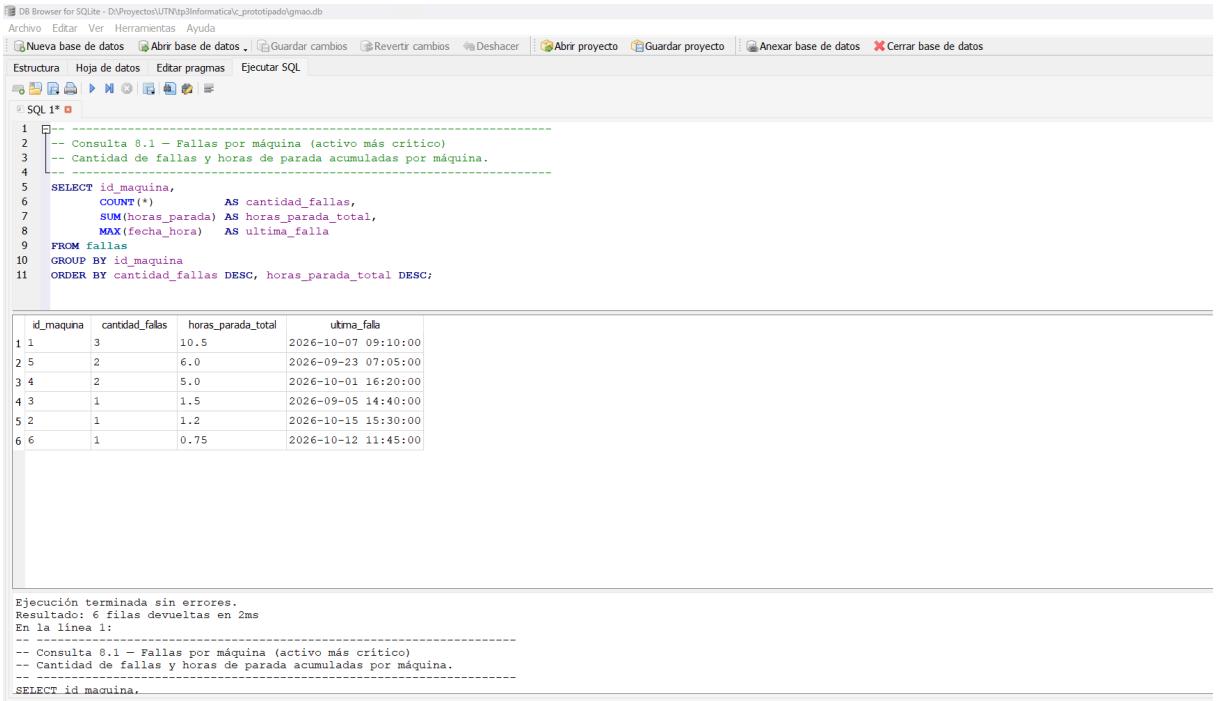


Figura 22. Consulta 8.1 en Ejecutar SQL: fallas y horas de parada por máquina

Resultado: la máquina 1, el torno CNC, es el activo más crítico: 3 fallas y 10,5 horas de parada, todas mecánicas en el husillo. Le siguen la prensa hidráulica (máquina 5) y el robot de soldadura (máquina 4), con 2 fallas cada uno; la prensa queda primero por tener más horas de parada (6 contra 5). El resultado muestra el identificador de la máquina: en el Paso 9 se resuelve el nombre con un `JOIN`.

Consulta 8.2 — Horas-hombre por tipo de mantenimiento

Necesidad: comparar el esfuerzo dedicado al mantenimiento correctivo y al preventivo. Una planta con mucho correctivo trabaja apagando incendios; el objetivo es desplazar horas hacia el preventivo.

```
-- ------------------------------------------------------------------
-- Consulta 8.2 — Horas-hombre por tipo de mantenimiento
-- Órdenes emitidas y horas-hombre registradas, correctivo vs preventivo.
-- ------------------------------------------------------------------
SELECT tipo,
       COUNT(*)                    AS cantidad_ot,
       SUM(horas_hombre)           AS horas_hombre_total,
       ROUND(AVG(horas_hombre), 2) AS horas_hombre_promedio,
       MAX(horas_hombre)           AS horas_hombre_maxima
FROM ordenes_trabajo
GROUP BY tipo
ORDER BY horas_hombre_total DESC;
```

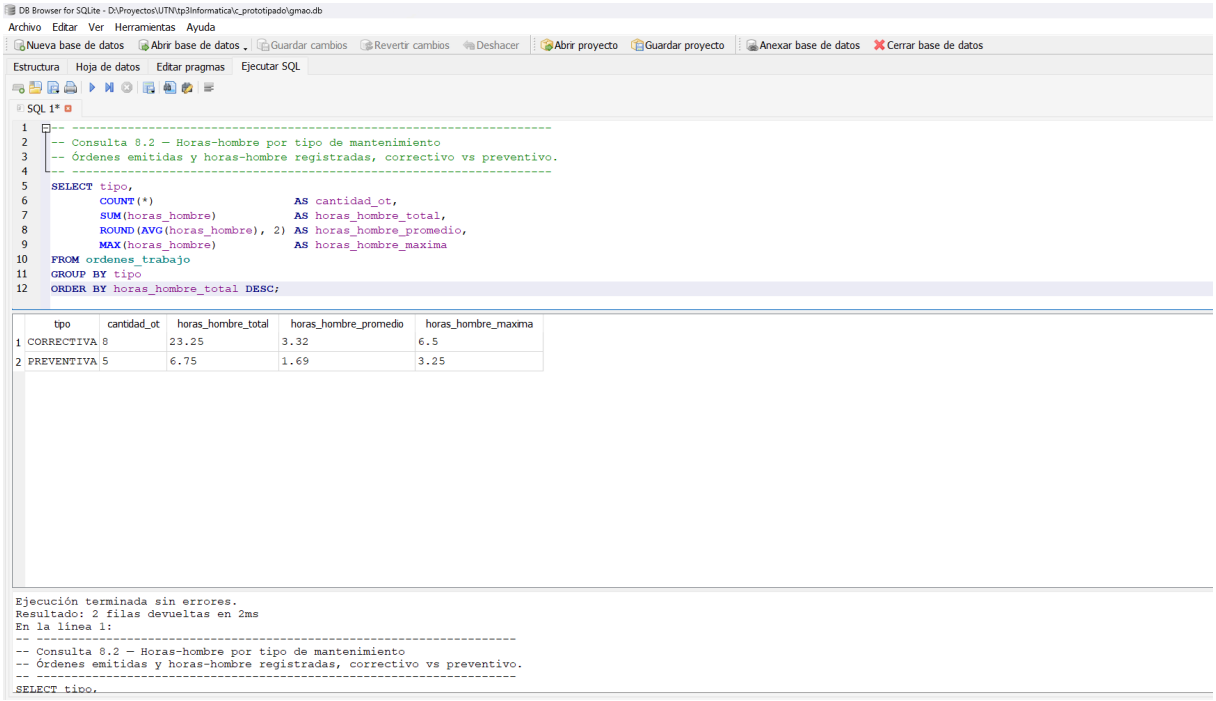


Figura 23. Consulta 8.2 en Ejecutar SQL: órdenes y horas-hombre por tipo de mantenimiento

Resultado: el correctivo concentra 8 órdenes y 23,25 horas-hombre, el 77,5 % de las 30 horas registradas; el preventivo, 5 órdenes y 6,75 horas. El promedio por orden correctiva (3,32 h) es casi el doble del de la preventiva (1,69 h). Los promedios se calculan sobre las

órdenes con horas cargadas: 7 de las 8 correctivas y 4 de las 5 preventivas, porque las dos pendientes tienen `horas_hombre` en `NULL`.

Paso 9 — Consultas de integración de tablas (JOINs)

Justificación teórica

En un modelo normalizado cada dato está en una sola tabla y las demás lo referencian por su clave. Una orden de trabajo guarda `id_maquina = 1`, no el texto «Torno CNC». Para mostrar la información completa hay que **reunir** las tablas siguiendo las relaciones PK–FK del DER, con `JOIN … ON`:

- `INNER JOIN` devuelve solo las filas que tienen pareja en las dos tablas.
- `LEFT JOIN` devuelve todas las filas de la tabla de la izquierda; si no tienen pareja, las columnas de la derecha quedan en `NULL`.

Los **alias** de tabla (`ordenes_trabajo AS ot`) acortan el código e indican de qué tabla sale cada columna cuando dos tablas tienen columnas con el mismo nombre, como `id_maquina`. El operador `||` concatena texto; si alguno de sus operandos es `NULL`, el resultado completo es `NULL`. `COALESCE(a, b)` devuelve el primer valor que no sea `NULL`.

Procedimiento

Se siguió el mismo procedimiento que en los Pasos 7 y 8. Antes de escribir cada consulta se tomó del DER el camino de claves entre las tablas a cruzar.

Consulta 9.1 — Órdenes con máquina, falla y técnico

Necesidad: el listado de órdenes que lee el personal de planta, con el nombre de la máquina, el motivo de la orden y el técnico asignado en lugar de los identificadores.

Máquina y técnico son obligatorios en toda orden, por eso se unen con `INNER JOIN`. La falla y el plan, en cambio, dependen del tipo de orden: la correctiva tiene falla y no tiene plan, y la preventiva al revés. Con `INNER JOIN fallas` desaparecerían las 5 órdenes preventivas; con `LEFT JOIN` se conservan todas y `COALESCE` muestra, en una sola columna, la descripción de la falla o la tarea del plan.

```
-- ---------------------------------------------------------------------
-- Consulta 9.1 — Órdenes con máquina, falla y técnico
-- Para las correctivas se muestra la falla y para las preventivas, la
-- tarea del plan. Una OT que atiende dos fallas aparece dos veces.
-- ---------------------------------------------------------------------
SELECT ot.nro_ot,
       ot.tipo,
       m.nombre                          AS maquina,
       COALESCE(f.descripcion,
                'Plan ' || p.codigo || ': ' || p.tarea) AS motivo,
       t.apellido || ', ' || t.nombre    AS tecnico,
       ot.estado
FROM ordenes_trabajo AS ot
INNER JOIN maquinas AS m          ON m.id_maquina = ot.id_maquina
INNER JOIN tecnicos AS t          ON t.id_tecnico = ot.id_tecnico
```

```
LEFT JOIN fallas AS f                ON f.id_ot = ot.id_ot
LEFT JOIN planes_preventivos AS p  ON p.id_plan = ot.id_plan
ORDER BY ot.nro_ot, f.fecha_hora;
```

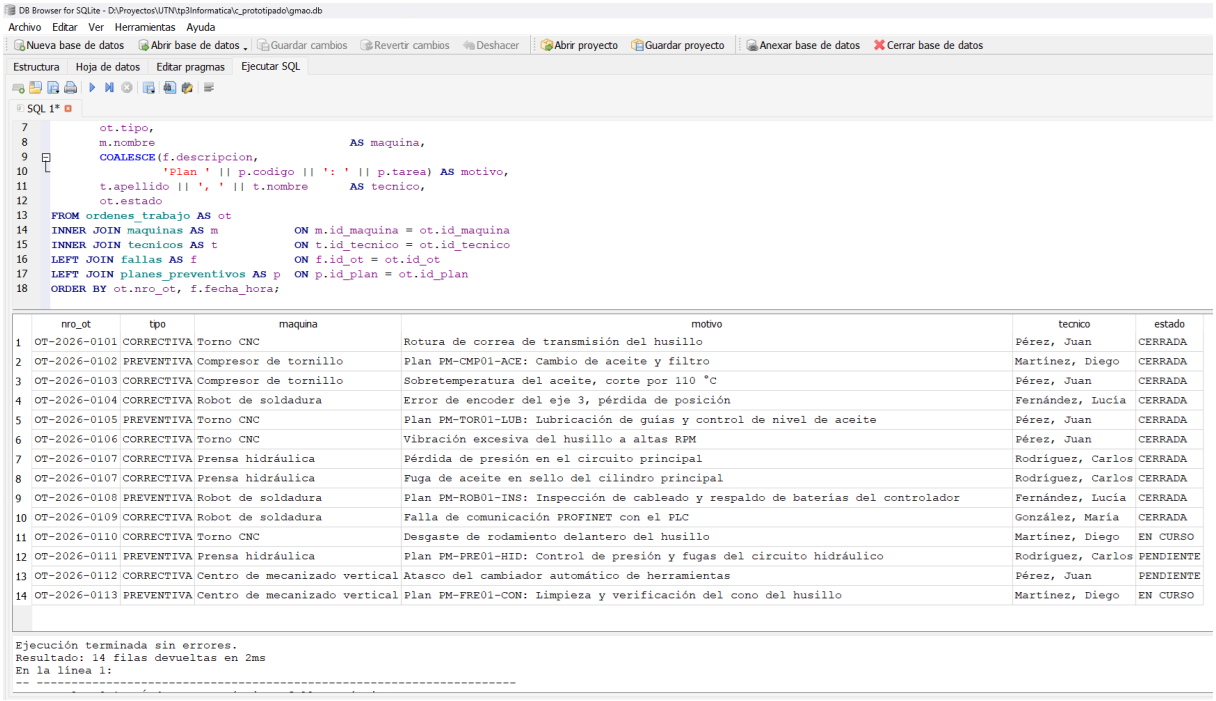


Figura 24. Consulta 9.1 en Ejecutar SQL: 14 filas con máquina, motivo y técnico de cada orden

Resultado: 14 filas para las 13 órdenes, porque la OT-2026-0107 atiende dos fallas de la prensa hidráulica (pérdida de presión y fuga en el sello) y aparece una vez por cada una. La falla pendiente de la inyectora no figura porque todavía no tiene orden: la consulta parte de las órdenes.

Consulta 9.2 — Repuestos consumidos por orden de trabajo

Necesidad: conocer el costo en repuestos de cada intervención, para imputarlo a la máquina y controlar el gasto de mantenimiento. La consulta parte de la tabla intermedia `ot_repuestos`, que resuelve la relación N:M, y la une con la orden, la máquina y el repuesto. El costo de cada consumo se calcula como cantidad por costo unitario.

```
-- ------------------------------------------------------------------------
-- Consulta 9.2 — Repuestos consumidos por orden de trabajo
-- Con la máquina, la cantidad y el costo de cada consumo.
-- ------------------------------------------------------------------------
SELECT ot.nro_ot,
       m.nombre                          AS maquina,
       r.descripcion                     AS repuesto,
       otr.cantidad,
       r.costo_unitario,
       otr.cantidad * r.costo_unitario AS costo_total
FROM ot_repuestos AS otr
INNER JOIN ordenes_trabajo AS ot ON ot.id_ot = otr.id_ot
INNER JOIN maquinas AS m         ON m.id_maquina = ot.id_maquina
INNER JOIN repuestos AS r        ON r.id_repuesto = otr.id_repuesto
ORDER BY ot.nro_ot, r.descripcion;
```

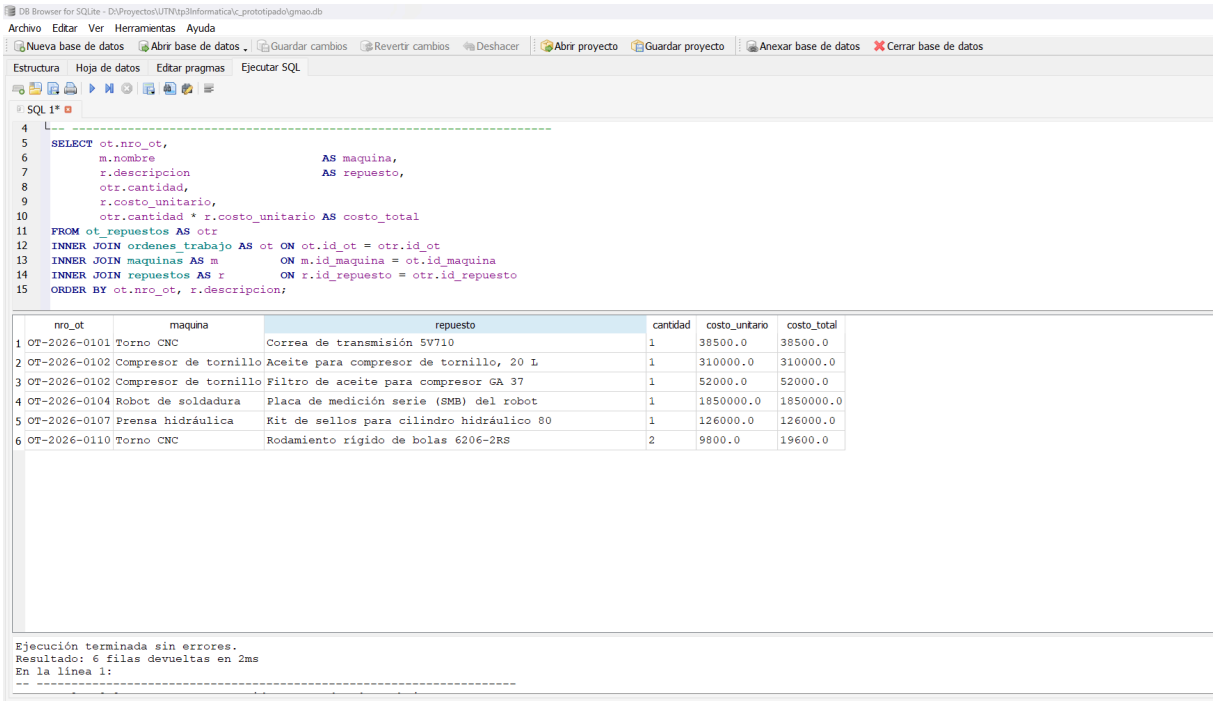


Figura 25. Consulta 9.2 en Ejecutar SQL: repuestos consumidos por orden, con máquina y costo

Resultado: seis consumos por un total de $ 2.396.100. La placa SMB del robot de soldadura ($ 1.850.000) explica el 77 % del gasto, y su stock quedó en cero (consulta 7.2): conviene reponerla antes de una nueva falla del robot.

Referencias

Fuentes consultadas el 19 de octubre de 2026. Las de proveedores se usaron solo para definiciones generales.

[1] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. https://doi.org/10.1145/362384.362685

[2] Amazon Web Services. *What is a relational database?* https://aws.amazon.com/relational-database/

[3] IBM. *What is a NoSQL database?* https://www.ibm.com/think/topics/nosql-databases

[4] InfluxData. *Relational databases vs time series databases.* https://www.influxdata.com/blog/relational-databases-vs-time-series-databases/

[5] dataPARC. *Time-series database vs. relational database vs. data lake.* https://www.dataparc.com/blog/time-series-database-vs-relational-database-vs-data-lake/

[6] International Society of Automation. *ISA-95 Standard: Enterprise-Control System Integration.* https://www.isa.org/standards-and-publications/isa-standards/isa-95-standard

[7] TeepTrak. *MES architecture ISA-95 implementation 2026.* https://teeptrak.com/en/mes-architecture-isa-95-implementation-2026/

[8] Wikipedia. *ISA-88.* https://en.wikipedia.org/wiki/ISA-88

[9] Accendo Reliability. *CMMS in maintenance.* https://accendoreliability.com/cmms-in-maintenance/

[10] SQLite. *About SQLite.* https://www.sqlite.org/about.html

[11] SQLite. *Appropriate Uses For SQLite.* https://www.sqlite.org/whentouse.html

[12] DB Browser for SQLite. Sitio oficial. https://sqlitebrowser.org/

[13] sqlitebrowser. *DB Browser for SQLite* (repositorio en GitHub). https://github.com/sqlitebrowser/sqlitebrowser

[14] DB Browser for SQLite. *Downloads.* https://sqlitebrowser.org/dl/

[15] SQLite. *SQLite Foreign Key Support.* https://www.sqlite.org/foreignkeys.html

[16] SQLite. *Datatypes In SQLite.* https://www.sqlite.org/datatype3.html